Budapesti Műszaki és Gazdaságtudományi Egyetem
Villamosmérnöki és Informatikai Kar
Irányítástechnika és Informatika Tanszék

SZAKDOLGOZAT FELADAT

Gaál Botond

BSc mérnök-informatikus hallgató részére

Pozícióalapú folyadékszimuláció

A pozícióalapú folyadékszimuláció a Smoothed Particle Hydrodynamics (SPH) és a Position Based Dynamics (PBD) módszerek együttese. Az SPH a számítógépes grafikában széles körben elterjedt módszer, mely a folyadék (vagy gáz) részecskék befolyását egy kernelfüggvény szerint teríti ki a térben. Koncepcionális egyszerűsége vonzó, és a modern megvalósításai képesek az összenyomhatatlanság kikényszerítésére, bár rendkívül számításigényesen. A PBD a fizikai szimulációnak olyan formalizmusa, amely a belső erők hatásának integrálása helyett azokat pozíciófüggő kényszerekkel írja le, és ezen kényszereket oldja meg Jacobi-iterációval.

A pozícióalapú folyadékszimuláció (Position Based Fluids, PBF) a PBD módszer összenyomhatatlan folyadékokra megfogalmazása, ahol a belső kényszerek ezen összenyomhatatlanságot hivatottak fenntartani, míg a folyadék részecskéit SPH-val megegyező módon definiálja, valamint a folyadéktérben értelmezett tulajdonságokat, mint a sűrűség, SPH módszerrel számolja.

A konkrét implementáció sarkalatos pontja a szomszédos részecskék keresésének hatékony megoldása, jellemzően valamilyen térbeli hash implementációra alapozva. Ez közvetlenül megnyitja az utat a GPU-s párhuzamosítás előtt.

A szakdolgozat a következőket tárgyalja:

- Tanulmányozza és ismertesse a PBF módszer alapjait képező irodalmat (SPH és PBD módszerek), és annak matematikai, fizikai hátterét!
- Tekintse át és ismertesse a hatékony megvalósítás nehézségeit, és lehetséges megoldásait a szakirodalomban!
- Tervezze és valósítsa meg a PBF-szimulátor alaprendszerét: részecskék állapotreprezentációját, szomszédsági keresést, a kényszerek iteratív megoldását, tartályfal határfeltételeket!
- Valósítsa meg a szimuláció GPU-s gyorsítását DirectX 12 compute shaderek segítségével!
- Vizualizálja a szimulációt (pl. pontfelhő- vagy billboard alapú megjelenítéssel)!
- Mutassa be a rendszer működését reprezentatív teszteseteken (állóvíz, hullámzás, akadály körüli áramlás)!
- Ismertesse a megvalósítás eszközeit, és értékelje az elkészült rendszert!

Tanszéki konzulens: Dr. Szécsi László, egyetemi docens

HALLGATÓI NYILATKOZAT

Alulírott **Gaál Botond**, szigorló hallgató kijelentem, hogy ezt a szakdolgozatot meg nem engedett segítség nélkül, saját magam készítettem, csak a megadott forrásokat (szakirodalom, eszközök stb.) használtam fel. Minden olyan részt, melyet szó szerint, vagy azonos értelemben, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem. A Függelékben egyértelműen megjelöltem, hogy a mesterséges intelligencia eszközeit alkalmaztam-e a dolgozat elkészítéséhez; amennyiben igen, annak módját és mértékét a táblázatban közöltem. Tudomásul veszem, hogy a mesterséges intelligenciával generált tartalomért – annak mértékétől függetlenül – teljes felelősséggel tartozom.

Hozzájárulok, hogy a jelen munkám alapadatait (szerző(k), cím, angol és magyar nyelvű tartalmi kivonat, készítés éve, konzulens(ek) neve) a BME VIK az interneten nyilvánosan hozzáférhetővé tegye, a munka teljes szövege pedig a BME Címtárban regisztrált személyek számára elérhető legyen. Kijelentem, hogy a benyújtott munka és annak elektronikus verziója megegyezik. Dékáni engedéllyel titkosított diplomatervek esetén a dolgozat szövege csak 3 év eltelte után válik hozzáférhetővé.

Kelt: Budapest, 2026. 05. 29.

...…….

Gaál Botond

Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Control Engineering and Information Technology

Gaál Botond

POSITION BASED FLUIDS

SUPERVISOR

Dr. Szécsi László

BUDAPEST, 2026

Összefoglaló

Jelen szakdolgozat témája egy valós idejű pozícióalapú folyadékszimuláció (Position Based Fluids, PBF) tervezése és implementációja. A PBF a Smoothed Particle Hydrodynamics-et (SPH) kombinálja a Position Based Dynamics (PBD) keretrendszerrel, az összenyomhatatlanságot a pozíciókon megfogalmazott sűrűség kényszerrel reprezentálva. Ez a megközelítés magával hozza a részecske alapú eljárások rugalmasságát és a PBD feltétel nélküli stabilitását, lehetővé téve a nagy időlépések és kicsi SPH kernel sugarak használatát. Ezen tulajdonságok alkalmassá teszik olyan valós idejű alkalmazásokban való felhasználásra, mint a videójátékok, vagy interaktív animációk.

A szimuláció C++ nyelven íródott, valamint HLSL shaderek felhasználásával, melyek a DirectX 12 API-t célozzák. Egy double-bufferelt, dual-queue architektúra átlapolja a fizikai számítást és a renderelést a teljesítmény maximalizálása végett. A memória térbeli koherenciáját egy Morton-kód szerinti számláló rendezés tartja fenn, melyet tovább gyorsít egy párhuzamos prefix-összeg számítás. A solver örvényesség-megerősítést és viszkozitást is beépít a vizuálisan gazdag folyadékviselkedés eléréséhez, valamint megvalósítja az adaptív PBF kiegészítést, mely a solver iterációit az egyes részecskék vizuális hozzájárulása alapján osztja szét. A folyadékmegjelenítés ray marching segítségével azonosítja a folyadékfelszínt egy 3D sűrűségmező alapján.

A tesztelés alapján ~40 fps érhető el NVIDIA RTX 4050 Laptop és NVIDIA RTX 3080 hardveren rendre 400,000 és 800,000 részecskével, még szélsőséges felhasználói beavatkozás esetén is, megerősítve, hogy a PBF hatékony és látványos megközelítés valós idejű folyadékszimulációhoz.

Abstract

This thesis presents the design and implementation of a real-time fluid simulation based on the Position Based Fluids (PBF) method. PBF combines Smoothed Particle Hydrodynamics (SPH) with the Position Based Dynamics (PBD) framework, formulating incompressibility as a positional density constraint. This approach inherits the flexibility of particle-based methods and the unconditional stability of PBD, enabling large time steps and small smoothing radii – properties that make it well suited for real-time applications such as video games and interactive animations.

The simulation is implemented in C++ with HLSL shaders targeting the DirectX 12 API. A double-buffered, dual-queue architecture overlaps physics computation and rendering to maximize throughput. Spatial coherence is maintained through a counting sort on a Morton-coded grid, accelerated by a parallel prefix scan. The solver incorporates vorticity confinement and viscosity to produce visually rich fluid behavior, and adopts the Adaptive PBF extension to distribute solver iterations according to each particle's visual contribution. Fluid rendering is achieved through ray-marched isosurface extracted from a splatted 3D density field.

Testing on NVIDIA RTX 4050 Laptop and NVIDIA RTX 3080 hardware demonstrated stable simulation at 40 fps using 400,000 and 800,000 particles respectively, even under extreme user-induced perturbations, validating PBF as a practical and visually compelling approach for real-time fluid simulation.

Table of Contents

1 Introduction

The goal of this thesis is the design and implementation of a real-time fluid simulation using the Position Based Fluids (PBF) method. I aim to demonstrate that this framework can be used to produce visually compelling results even in complex, dynamic, user-driven scenarios.

Fluid simulation in interactive applications such as video games presents a particular challenge: the simulation must run at a minimum of 30 fps and remain stable under unpredictable, user-induced conditions, while strict physical accuracy is only a secondary concern.

The established particle-based fluid simulation method, Smoothed Particle Hydrodynamics (SPH) [2], is well suited to interactive applications due to its flexibility and parallelizability, as demonstrated by Müller et al. [3]. However, stability in SPH requires small time steps, large smoothing radii and careful tuning of the stiffness parameter [1], which makes it difficult to use in the unpredictable conditions of interactive applications. Several other methods build on traditional SPH, with the aim to address these issues, such as Predictive-Corrective Incompressible SPH [5].

PBF addresses the issues of SPH by combining it with the Position Based Dynamics (PBD) framework [6], formulating incompressibility as a positional density constraint. This yields a solver that is unconditionally stable and tolerates large time steps, at the cost of physical accuracy: a worthwhile trade-off in interactive 3D applications.

This thesis is structured as follows: Chapter 2 surveys the relevant literature. Chapter 3 summarizes the theoretical foundations of PBF. Chapter 4 describes the algorithm and its GPU implementation. Chapter 5 explores attempts at further optimization, while Chapter 6 discusses performance and presents the results of the thesis.

2 Literature overview

The theoretical foundation of particle-based fluid simulation lies in Smoothed Particle Hydrodynamics (SPH), originally invented for use in astrophysical simulations. J. J. Monaghan's comprehensive review of the method [2] formalized its core principles:

the fluid is represented by particles, whose attributes are interpolated using a smoothing kernel. This eliminates the need for a fixed grid, making it well suited for scenes with complex boundaries.

Müller et al. [3] demonstrated that SPH can simulate fluids with free surfaces at interactive frame rates. By deriving pressure and viscosity forces from the Navier-Stokes equations and introducing purpose-built smoothing kernels, they achieved stable real-time simulations with particle count in the thousands. Their work also addressed surface tracking and rendering via point splatting and marching cubes.

Some limitations of force-based approaches, of which SPH is one, include their sensitivity to overshoots and reliance on small time steps to maintain stability. Müller et al. address this by introducing Position Based Dynamics (PBD) [6], a framework that operates directly on particle positions rather than forces or velocities. It iteratively projects particles to positions that better satisfy constraints which describe internal forces. PBD achieves unconditional stability and allows large time steps, at the cost of accuracy.

Macklin & Müller combined these two lines of work in the Position Based Fluids (PBF) method [1], which is the subject of this thesis. By formulating incompressibility as a positional density constraint in the PBD framework, and using standard SPH estimation for said density, PBF inherits the strengths of both methods. Macklin & Müller further describe ways of including vorticity confinement, viscosity and surface tension into the framework. These contributions allowed PBF to be used in real-time fluid simulations with more than a hundred thousand particles.

Köster & Krüger further improve PBF by introducing a lightweight extension to the method that adaptively adjusts the constraint solver iteration count of particles based on their contribution to the visual fidelity of the scene [4]. This can significantly lower the computational requirements of the simulation while preserving the visuals.

Hoetzlein showed how a counting sort on a spatial grid can be used to greatly improve spatial coherence of particle data [8]. This is demonstrated in the Fluids project which interactively simulates over 1 million particles with SPH using the CUDA Driver API [7].

3 Position Based Fluids

In this chapter, the theoretical foundations of the Position Based Fluids method are presented. The content follows the work of Macklin & Müller [1], but is expanded upon and reformulated with the goal of providing a more detailed, intuitive mathematical explanation.

3.1 External forces: semi-implicit Euler

The position based fluids approach updates the positions of the particles in two steps. First, based on the external forces (e.g. gravity) and the current velocity of a particle, we estimate its velocity for the next simulation step:

$$v_i \Leftarrow v_i + \Delta t \cdot f_{\text{ext}}(x_i) \tag{1}$$

Where v and x denote velocity and position, i indexes the simulated particles, Δt is the timestep for the given simulation step, and $f_{ext}(x_i)$ is the sum of external forces at the position of particle *i*. Note that this equation assumes that the mass of the particle is 1, which is an assumption that also applies for the density calculation. With the new estimated velocity, we can produce x_i^*, the prediction for the next position:

$$x_i^* \Leftarrow x_i + \Delta t \cdot v_i \tag{2}$$

In short, we predict the next position of the particle using a semi-implicit Euler step. However, this position does not take into account solid collisions, nor internal forces, which are represented by PBD constraints.

3.2 Internal forces: constraint projection

In PBD, internal forces are described using constraints. In our case, to enforce constant ρ_0 density (incompressibility), we define a C_i constraint function for the *i*th particle:

$$C_i(x_1, \ldots, x_n) = \frac{\rho_i}{\rho_0} - 1 \tag{3}$$

Where *n* is the number of particles in the simulation. This is a scalar function of all the simulation particles' positions, and it's constructed such that being equal to 0 means that the density at particle *i*'s position is the target density. We seek corrections for the

predicted position that move the particle to a new position that better satisfies C_i, bringing it closer to 0. For calculating ρ_i, we borrow the core idea from SPH: instead of a particle having an exact boundary, its influence is spread out around its position according to a smoothing kernel. In this thesis, we follow Macklin & Müller, and use the following two kernels [9]:

$$W_{\text{poly6}}(r,h) = \frac{315}{64\pi h^9}\begin{cases}(h^2 - r^2)^3 \; if \; 0 \le r \le h \\ 0 \; otherwise\end{cases} \tag{4}$$

$$\nabla W_{\text{poly6}}(r,h) = \frac{315}{64\pi h^9}\begin{cases}-6r(h^2 - r^2)^2 \; if \; 0 \le r \le h \\ 0 \; otherwise\end{cases} \tag{5}$$

$$W_{\text{spiky}}(r,h) = \frac{15}{\pi h^6}\begin{cases}(h - r)^3 \; if \; 0 \le r \le h \\ 0 \; otherwise\end{cases} \tag{6}$$

$$\nabla W_{\text{spiky}}(r,h) = \frac{-45}{\pi h^6}\begin{cases}\frac{r}{|r|}(h - r)^2 \; if \; 0 \le r \le h \\ 0 \; otherwise\end{cases} \tag{7}$$

Where *h* is the smoothing kernel radius, outside of which the influence of a particle is 0, so that when considering the neighbors of a particle, we only need to look inside a sphere with radius *h*. According to SPH, the density at particle *i* is as follows:

$$\rho_i = \sum_j W_{\text{poly6}}(x_i - x_j, h) \tag{8}$$

In this equation, once again we assume that all particles have mass 1.

In PBD, we aim to find a particle position correction vector $\Delta x = [\Delta x_1, \dots, \Delta x_n]$ that, when added to the position vector $x = [x_1, \dots, x_n]$ better satisfies all C_i constraints:

$$C_i(x + \Delta x) = 0 \tag{9}$$

To this end, we move the particles along the gradient of the constraint function:

$$\Delta x = \nabla C_i(x)\lambda_i \tag{10}$$

Where λ_i is the length of the Newton step along the gradient for constraint *i*. We approximate the constraint function using its first-order Taylor expansion around x:

$$C_i(x + \Delta x) \approx C_i(x) + \nabla C_i^T \cdot \Delta x = 0 \tag{11}$$

Substituting equation 10 into equation 11, we get:

$$C_i(x + \Delta x) \approx C_i(x) + \nabla C_i^T \nabla C_i \lambda_i = 0 \tag{12}$$

Solving for λ_i gives:

$$\lambda_i = \frac{-C_i(x)}{\nabla C_i^T \nabla C_i} \tag{13}$$

Let us introduce the notation $\nabla_{x_i} C_i$ to refer to the gradient of the function we get by fixing all inputs of C_i except the position of particle *i*. Let us call this the *gradient with respect to a specific particle's position*. Using this notation, the previous equation can be written as:

$$\lambda_i = \frac{-C_i(x)}{\sum_j \left|\nabla_{x_j} C_i(x_j)\right|^2} \tag{14}$$

The gradient in the denominator is the gradient of a function defined on the particles, which in the SPH formulation, can be given as:

$$\nabla_{x_j} C_i(x_j) = \frac{1}{\rho_0} \sum_k \nabla_{x_j} W(x_i - x_k, h) \tag{15}$$

With the subscript of ∇ showing whether we consider *i* or *j* to be the input. This can be deconstructed into two different cases depending on whether or not *j*=*i*. In other words, we can look at how the constraint changes with its own particle's position, or we can look at how the constraint changes with neighboring particles' positions (SPH estimator for the gradient of a function defined on the particles):

$$\nabla_{x_j} C_i(x_j) = \frac{1}{\rho_0} \begin{cases} \sum_{k \neq i} \nabla_{x_i} W_{\text{spiky}}(x_i - x_k, h) \;\; if\; j = i \text{ (own particle)} \\ -\nabla_{x_j} W_{\text{spiky}}(x_i - x_j, h) \;\; if\; j \neq i \text{ (neighboring particle)} \end{cases} \tag{16}$$

When the denominator in equation 14 approaches 0, the simulation may become unstable. This is rectified using constraint force mixing (CFM), where an ε relaxation parameter is introduced, "softening" the constraints:

$$\lambda_i = \frac{-C_i(x)}{\sum_j \left|\nabla_{x_j} C_i(x_j)\right|^2 + \varepsilon} \tag{17}$$

Now that we have the recipe for calculating constraint gradients (equation 16), and can use them to calculate λ_i, all that's left is deriving the per-particle position updates:

$$\Delta x_i = \frac{1}{\rho_0} \sum_j (\lambda_i + \lambda_j) \nabla W_{spiky}(x_i - x_j, h) \tag{18}$$

3.3 Tensile instability

Whenever a particle has too few neighbors, density requirements cannot be satisfied. This leads to such particles forming tight clumps that futilely attempt to achieve the desired density. This can be rectified by not allowing negative pressures; however, this degrades particle cohesion as well as completely disabling surface tension. The solution used by Macklin & Müller, originally proposed by Monaghan [10] is also what this thesis implements. It adds a purely repulsive term to the smoothing kernel that aims to keep the density slightly lower than the target:

$$s_{\text{corr}} = -k\left(\frac{W(x_i - x_j, h)}{W(\Delta q, h)}\right)^n \tag{19}$$

Where k, Δq and n are tunable parameters. In practice, we leave $\Delta q = 0.2h$ and $n = 4$ as sensible defaults, only tuning k. Using this corrective factor s_{corr}, we can include this artificial pressure in the position update in the following way:

$$\Delta x_i = \frac{1}{\rho_0}\sum_j \left(\lambda_i + \lambda_j + s_{\text{corr}}\right)\nabla W_{\text{spiky}}(x_i - x_j, h) \tag{20}$$

3.4 Vorticity confinement

Position based methods suffer from unwanted dissipation of energy, leading to undesirable damping. In the case of fluid simulations, this makes the fluid movement sluggish and less turbulent. The solution used by Macklin & Müller is introducing vorticity confinement, in two steps:

First, we calculate the vorticity ω_i (curl of the velocity vector field) at the particles' position, using the standard SPH estimation for the curl of a function defined on the particles:

$$\omega_i = \nabla \times v = \sum_j (v_j - v_i) \times \nabla_{x_j} W_{\text{spiky}}(x_i - x_j, h) \tag{21}$$

Once we have the vorticity, we can introduce an external force that aims to amplify it, by a tunable factor of ε:

$$\eta = \nabla|\omega|_i \tag{22}$$

$$f_i^{\text{vorticity}} = \varepsilon\left(\frac{\eta}{|\eta|} \times \omega_i\right) \tag{23}$$

These equations merit some explaining: the curl of the velocity field, i.e. the vorticity forms a vector field assigning a "spin" to each particle's spatial position. In the words of Fedkiw et al.: "Each small piece of vorticity can be thought of as a paddle wheel trying to spin the flow field in a particular direction. Artificial numerical dissipation damps out the effect of these paddle wheels, and the key idea is to simply add it back" [11]. $|\omega|$ in turn then is a scalar field assigning the magnitude of vorticity to each of those positions, while its gradient η points from areas with low vorticity to areas with higher vorticity. It gets normalized to produce a unit vector that we can use to decide which way to point the force that will attempt to "turn the paddle wheel".

3.5 Viscosity

Viscosity is, once again, governed by a tunable strength *c*, and is an extra pass that modifies velocities of particles to bring them closer to the velocity of neighboring particles in the following way:

$$v_i \Leftarrow v_i + c \sum_j (v_j - v_i) W_{\text{poly6}}(x_i - x_j, h) \tag{24}$$

4 Algorithm & Implementation

This section will go into detail about the algorithm this thesis uses and where it is relevant, talk about the implementation details. The simulation is implemented in C++, with compute shaders written in HLSL targeting the DirectX 12 API. This thesis assumes familiarity with GPU programming concepts and the DirectX 12 API, as a thorough introduction to these falls outside its scope, yet knowledge of them is required to follow implementation details.

4.1 Double buffered command queues

The simulation uses two command queues:

1. A "compute queue" running the physics calculations, which produce particle data for the graphics queue to consume. Since it runs compute shaders exclusively, its type is Compute.

2. A “graphics queue” displaying the scene based on the particle data calculated by the compute queue. Since it handles graphics commands, and auxiliary compute commands that do not modify particle data, its type is Direct.

These two command queues handle the display and compute in a double buffered fashion: the compute queue is the producer; the graphics queue is the consumer. While the compute queue is constructing the particle data for frame N into the back buffers, the graphics queue can display the scene based on the data found in the front buffers, which describe frame N-1. Each queue has its own fence, but both use a shared frame counter for synchronization in the render call. Render call N therefore does two things:

- On the compute queue: dispatches calculations for the particle data that will be shown in frame N.
- On the graphics queue: dispatches graphics commands that display frame N-1 to the screen.

The synchronization in render call N is as follows (see Figure 1):

1. CPU wait for the compute queue’s fence to reach N-1, signaling that the compute queue is done with calculating the data for frame N-1.
2. Flip the double buffered resources. This can only be done safely when we are assured that neither command queue is using the double buffered resources.
    a. For the compute queue, this is assured by the fact that during the production of frame N-1, the last command inserted was the compute fence signal for N-1. Since the CPU just waited for N-1, and no new commands have been inserted, the compute queue must be idle.
    b. As for the graphics queue: render call N, which dispatches calculations for N, and graphics for N-1 was preceded by render call N-1, which therefore dispatched calculations for N-1, and graphics for N-2. Therefore, to ensure that each render call results in exactly one frame’s particle data calculation and one frame’s display, render call N-1 may end only when frame N-2 is presented. This is ensured by signaling the graphics fence with N-2 in render call N-1, and immediately CPU waiting for it. This means that when the next render call happens, the graphics queue is idle, and we can safely swap the double buffers.

    c. Note that at this point, no GPU commands are in flight, meaning that non-double buffered GPU resources can be accessed safely. This includes constant buffers that change between frames.

3. Compute queue: dispatch the shaders that fill the back buffers with the particle data of frame N: this is the physics step.
4. Insert a compute fence signal to the compute queue with value N: when the compute queue reaches this fence, the particle data in the back buffers will represent frame N.
5. Graphics queue: GPU wait for the graphics fence to reach N-1, i.e. be done with calculating the data for frame N-1, which is the frame we're going to be displaying. Since the compute queue's runtime is much longer, this is almost always no-op, but is here for the edge case where the graphics takes longer than the compute.
6. Graphics queue: dispatch shaders that draw the scene based on the particle data in the front buffers.
    a. After these dispatches, the compute of frame N, and the graphics of frame N-1 are in flight at the same time, parallel with each other, as opposed to calculating and then displaying a frame sequentially, leading to higher fps at the cost of a 1-frame delay.
7. Insert a graphics fence signal to the graphics queue with value N-1: when the graphics queue reaches this fence, frame N-1 will have been presented to the screen. Immediately CPU-wait for this fence to be reached, so that we know the next render call may begin.

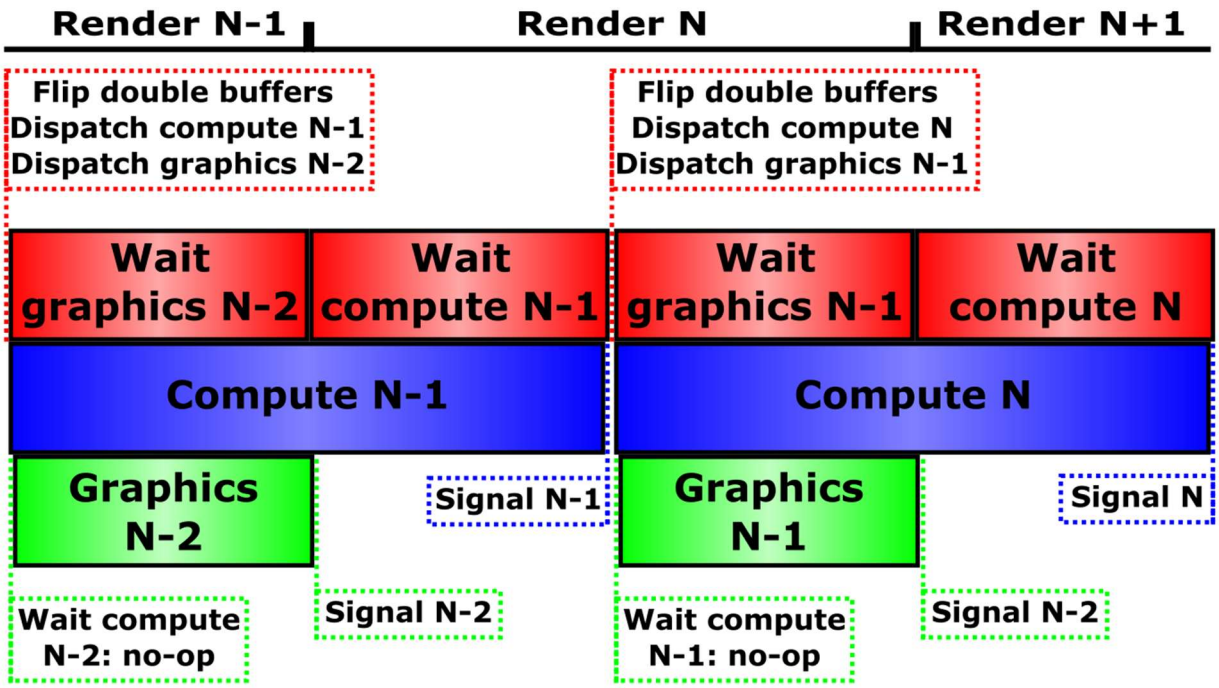

Figure 1: Compute-Graphics timeline.

4.2 Double buffer implementation

Particle data is initialized on the CPU, then uploaded to the GPU's default heap via an upload buffer. Particle data that must be accessed by both command queues has to be double buffered in order to be able to overlap compute and graphics. The specific implementation of double buffered GPU aims to avoid unnecessary copy commands. The GPU resources themselves (the actual allocated memory, and the data therein) are static, the "flipping" of the double buffer means swapping what the buffers' accessors (the objects through which we access the resources' data, whether they be GPU side handles or CPU side pointers) point to. To this end, the simulation handles two descriptor heaps:

- "static" descriptor heap: each GPU resource (buffer, texture) creates the descriptors it wants to expose (SRVs, UAVs, etc.) in this descriptor heap. This heap is non shader visible, its purpose is to be a single source of truth for descriptors.
- "main" descriptor heap: each shader reserves a contiguous area in the main (shader visible) heap, where it can store the descriptors it uses to access resources. Whenever a shader wants to access a resource, a descriptor gets copied to its allocated area in the main heap. This means that shaders that access the same resource might have duplicates of the same descriptor in their heap area.

When it comes to a double buffered resource, it shall have one slot for each of its buffers in the static heap. Flipping the double buffer, then, is only a matter of copying the correct descriptors from the static heap to the correct slots in the main heap. The double buffer keeps track of the shaders that want to access the front and the back buffers, and also keeps track of which underlying buffer is the front vs the back at a given time. When the buffer gets flipped, the double buffer re-routes accordingly.

For example, we might have a double buffered resource A, which has two descriptors in the static heap A0, and A1. Let us say that upon creation, A0 is the front, and A1 is the back buffer. We might have several shaders that want to use these resources in either way: for example, we might have shaders S1 and S2, which want to write to A's back buffer, and shaders S3 and S4 which want to read from A's front buffer. Swapping the buffers preserves the front-back assignments, only changing the routing of A0 vs A1, as seen on Figure 2.

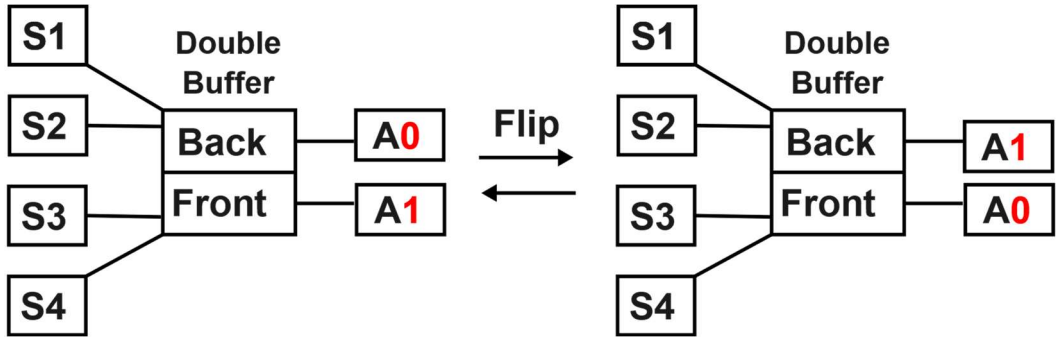

Figure 2: Flipping the double buffer. Note the unchanged S1-S4 Front vs Back assignments.

4.3 Physics step – Compute queue

As discussed, the simulation plays out via the interaction of the graphics and command queues, which synchronize using fences, and trade data via the double buffers. Now let us look at how the shader passes in the compute queue implement the earlier discussed PBF computation. The purpose of the physics step is to update particle data in the double buffer, getting it ready for the graphics queue to display to the screen. For particle i, its position x_i and velocity v_i are known before the physics run, which processes timestep Δt:

1. Predict next velocity and position based on external forces and collisions.
    . $v_i \Leftarrow v_i + \Delta t \cdot f_{\text{ext}}(x_i)$
    . $x_i^* \Leftarrow x_i + \Delta t \cdot v_i$

2. Assign the particles to grid cells based on their predicted position, and sort them according to their grid cell's index.
3. Perform collision response as a pre-stabilization step.
4. Calculate level of detail (LOD, i.e. iteration count).
5. Perform constraint projection & collision response LOD times (Chapter 4.3.4) per particle, resulting in a position that better satisfies density constraints.
    . LOD times: calculate $\lambda_i, \Delta x_i$ according to equations 17 and 20, then update $x_i^* \Leftarrow x_i^* + \Delta x_i$
6. Update velocity to reflect the new positions.
    . $v_i \Leftarrow \frac{1}{\Delta t}(x_i^* - x_i)$
7. Correct velocity to include vorticity confinement and viscosity.
8. Commit position: $x_i \Leftarrow x_i^*$

Let us examine each of these steps in detail, as well as some implementation decisions.

4.3.1 Prediction

The prediction step is a semi-implicit Euler step, as discussed in Chapter 3.1:

1. Provide a preliminary velocity prediction: $v_i \Leftarrow v_i + \Delta t \cdot f_{\text{ext}}(x_i)$. The external forces usually include gravity, although it can be disabled for an outer space simulation. It can also include any additional forces that lets the user interact with the liquid. In this thesis, two additional external forces have been implemented:
    a. A "raw" horizontal force, to slosh the liquid around (see Figure 5).
    b. A toggleable "fountain", which shoots a jet of liquid upwards by applying force to the particles only when they are within the bounds of the fountain (see Figure 4).
2. Apply solid collision response *for the velocity.* A collision response step is also part of the constraint projection loop: it moves new predicted positions out of solid bodies. Even though thematically the collision response regarding predicted position and velocity belongs together, they must be separate for the following reason: Once the predicted position is calculated based on the time step and the velocity (which has been amended using external forces), the velocity is not used

anymore for the given frame, and in fact gets overwritten once the position prediction gets corrected using the constraints. By applying solid collision response to the velocity *before* the constraint projection, its effect does not get overwritten. The velocity collision response consists of two effects as seen on Figure 3:

a. The velocity component perpendicular to the wall gets zeroed out.

b. The velocity component tangential to the wall gets reduced by a set percentage to mimic adhesion.

3. Supply $x_i^* \Leftarrow x_i + \Delta t \cdot v_i$ predicted position, which is now ready to be amended by the density constraints and solid collisions.

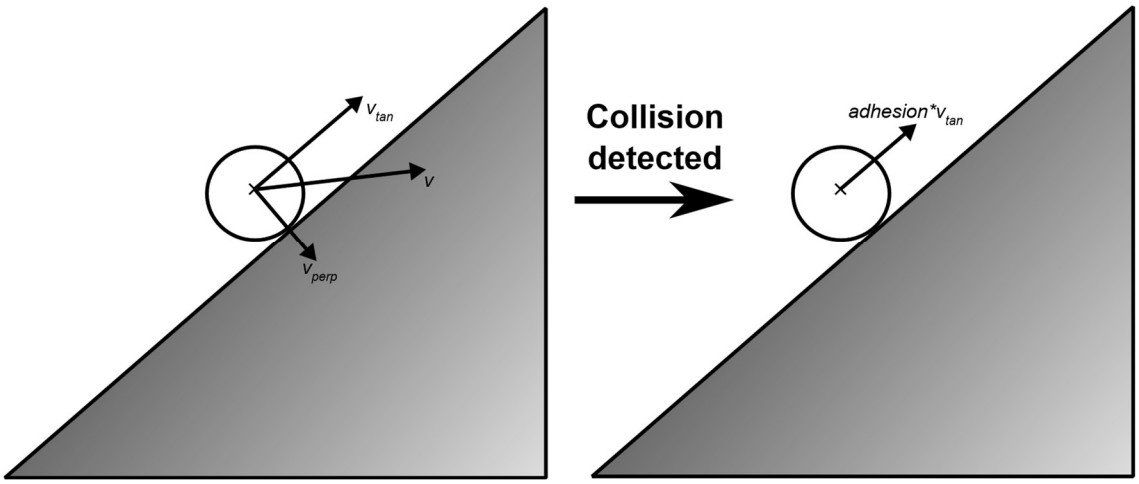

Figure 3: Collision velocity response.

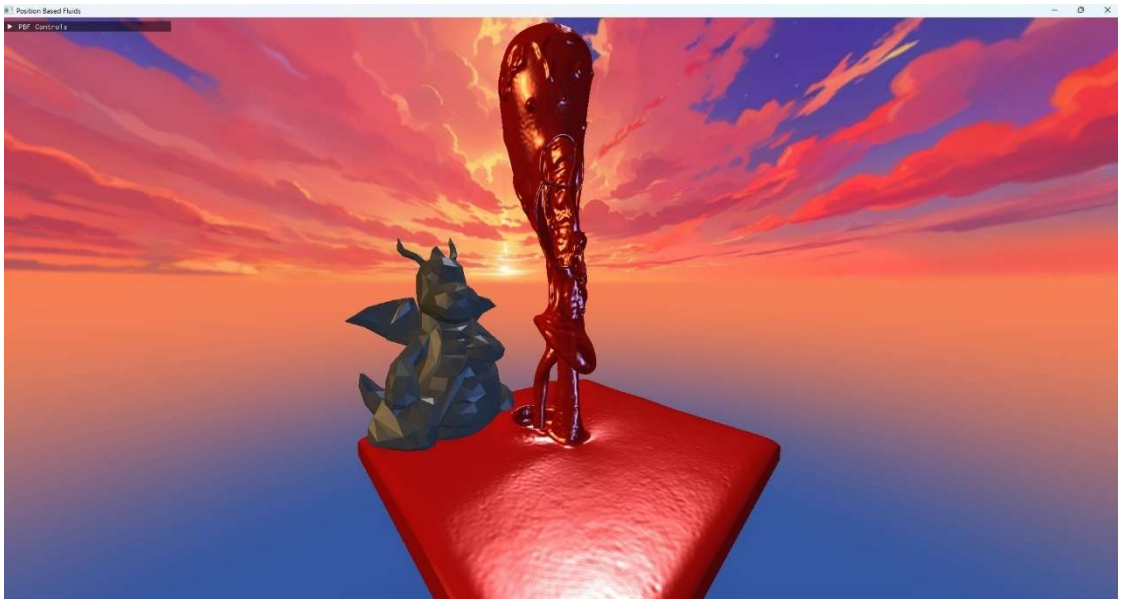

Figure 4: User-controlled forces in the simulation 1: fountain.

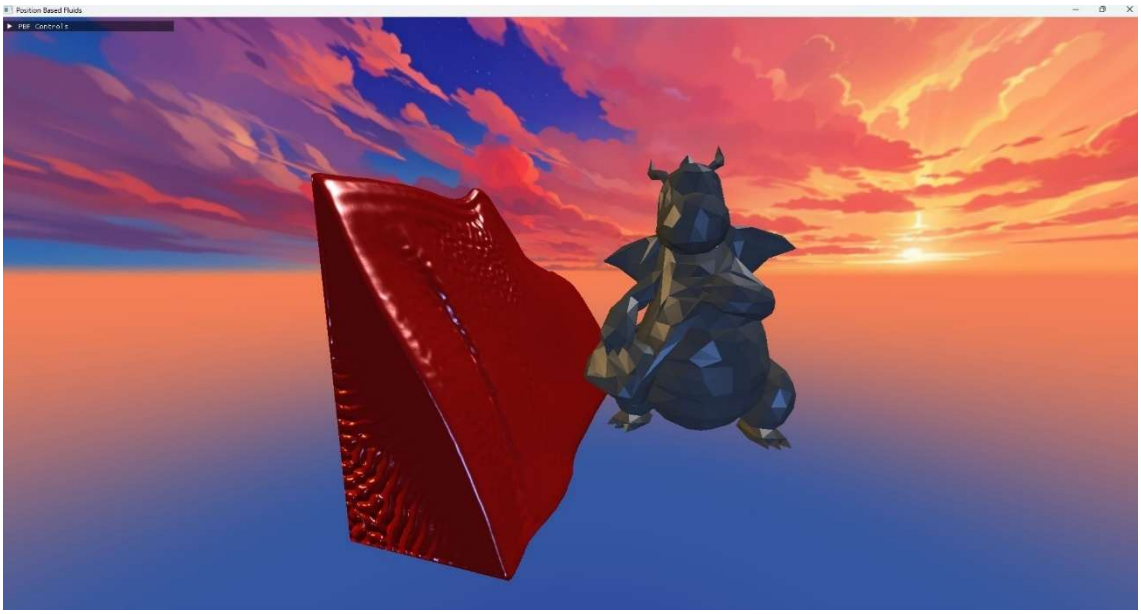

Figure 5: User-controlled forces in the simulation 2: external horizontal force.

4.3.2 Grid and Counting sort

Although the equations for PBF can be formulated such that they include all particles in the simulation, in practice, only neighboring particles influence each other. Two particles are neighbors if they are closer to one another than the radius of the SPH smoothing kernel. This is evident based on the fact that the SPH kernel functions are 0 outside of this boundary. To this end, an efficient neighbor lookup is needed to quickly determine which particles can be potential neighbors. This thesis uses a spatial grid: A particle belongs to exactly one grid cell based on its center point, identified by the cell's integer index idx_{cell}. Along each axis in the grid, these indices span from 0 to grid_size-1. The association between the world space coordinates of a particle and its index is: $idx_{cell} = clamp(floor(x_i - gridmin)/h, 0, grid\_size - 1)$, where *gridmin* is the world space position of the origin of the grid, i.e. the minimal corner of cell (0,0,0). Cells are cubes, which have a side length equal to the SPH smoothing radius. In choosing the cell size so, we guarantee that all neighbors of a particle can be found within the neighboring cells (whose integer index is no more than 1 off from the particle's cell index in either direction). This means that no matter how many cells are in the simulation, it is guaranteed that only 27 cells (3 indices per axis) need to be checked for neighboring particles.

The particles' data is stored in buffers on the default heap, in an order that is random unless the particles are sorted. Sorting the particles based on the cell they belong

to leaves particles in the same cell contiguous in memory (see Figure 6). Calculations are done one thread per particle, each thread gets assigned a particle by matching the particle's index in its buffer to the thread's dispatch ID. This means that threads inherit the particle's sorted nature: threads belonging to particles in the same cell will have consecutive dispatch IDs, and more often belong to the same Streaming Multiprocessor (SM), as well as the same warp.

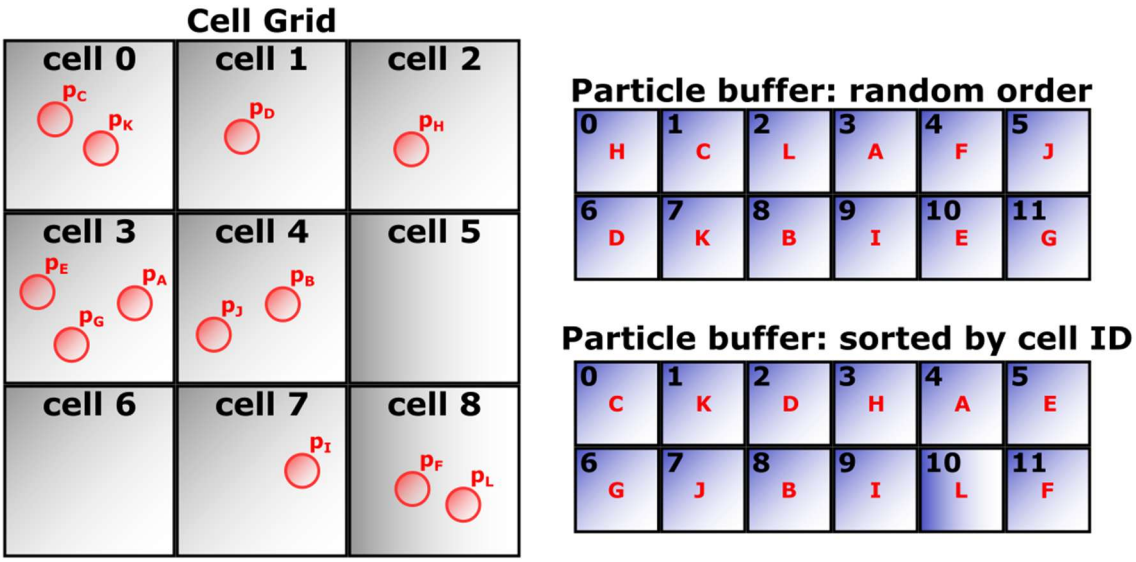

Figure 6: Unsorted vs sorted particle data in memory.

This then, is the core insight: sorting increases the chance of threads sharing L1 cache – or even belonging to the same warp – access the same or contiguous memory addresses. This increases the chance of the following:

1. Coalesced reads: When a particle's data must be fetched from VRAM, if all threads in a warp want to access data within a 128-byte block, the reads coalesce, leading to fewer lengthy memory transactions. For example, in Figure 6, threads 0-2 are assigned particles P_C, P_K, P_D – all residing in adjacent cells 0 and 1. When this warp needs to fetch their positions from VRAM, the requested addresses are contiguous (indices 0-2 in the buffer), so the hardware merges them into a single memory transaction. In the unsorted buffer, the same particles P_C, P_K, P_D sit at indices 1, 6, 7 – scattered across the buffer – forcing multiple separate transactions to fetch the same data.

2. Cache line sharing: Threads sharing L1 cache have an increased chance of automatically pre-loading each other's data by happening to be adjacent to data that threads in the same thread group need. Without sorting, each thread would

drag in cache lines that are mostly useless to their warp-mates. For example, in Figure 6, when thread 0 fetches P_C from the sorted buffer, the hardware pulls in a full cache line containing P_C, P_K, P_D… Threads 1 and 2 in the same warp then find their particles already in L1 – a free pre-load, since these are exactly the particles they were assigned. In the unsorted buffer, fetching P_C at index 1 pulls in a random assortment of particles P_H, P_L, etc. from cells 2, 8, etc. Most of these are useless to the warp's other threads.

3. Cache hit rate: Threads with particles belonging to the same cell will iterate over the same 27 neighbor cells. Now that these threads are more likely to share L1 cache, they'll be re-using the same set of cache lines repeatedly, rather than blowing up the working set of particles, increasing cache hit rate. For example, in Figure 6, threads 0 and 1 in the sorted buffer hold P_C and P_K, both in cell 0. Both threads iterate over the same cells (0,1,3,4,) reading particles like P_D (cell 1), P_E, P_G, P_A (cell 3) and P_B, P_J (cell 4). Since they share L1, the cache line holding P_E, P_G, P_A is fetched once and reused by both threads. In the unsorted case, threads 0 and 1 hold P_H (cell 2) and P_C (cell 0), which operate on a different set of neighbor cells, so each thread evicts the other's working set, decreasing cache hit rate.

The sort requires a cell ID to sort by, i.e. a 1D index, rather than the currently defined spatial 3D integer index. This 3D index to 1D index assignment offers another chance for optimization: instead of using simple row-major indexing, we may use a 1D index that offers better spatial locality: Morton Codes. Morton codes (also called Z-order codes) map multi-dimensional coordinates to a single integer by interleaving the bits of each coordinate, producing an ordering that preserves spatial proximity. This defines a space-filling curve, so cells with similar IDs are more likely to be spatially nearby than with row-major indexing. This means that in addition to particles within the same cell being memory contiguous, particles in different, spatially nearby cells will also not be far apart in memory. Figure 7 illustrates the spatial locality of Morton Codes.

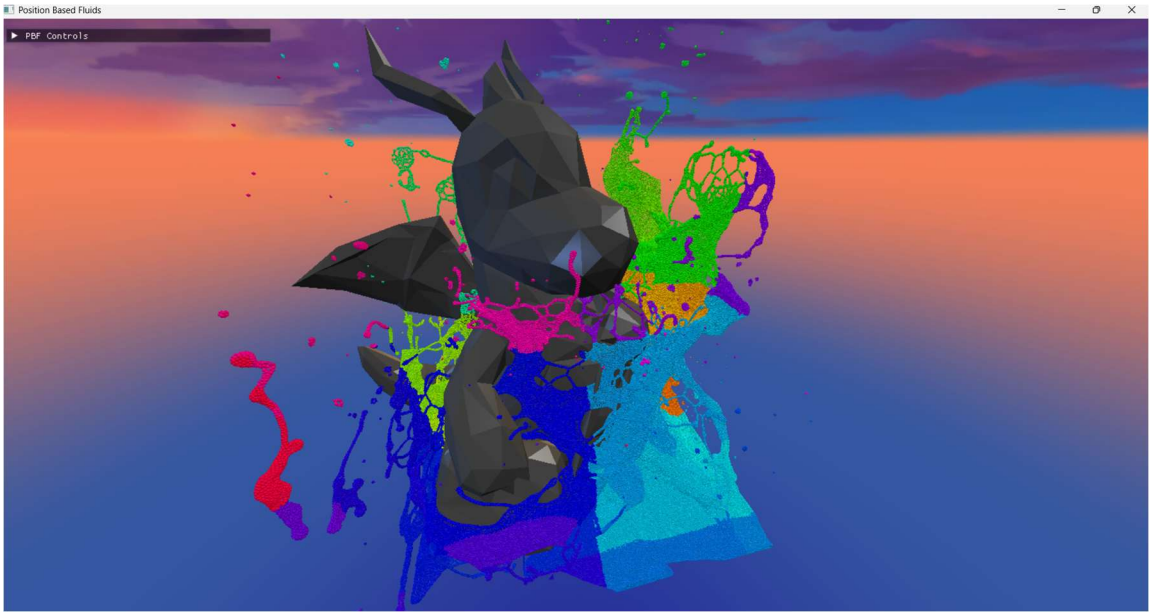

Figure 7: Visualization of Morton Codes. Similar colors mean similar cell IDs.

The sorting is done using a counting sort:

1. Calculate a particle count for each cell: $count[c]$
    a. Shader dispatched per-particle: each particle calculates its 1D cell index and atomically increments that cell's count.
2. Calculate an exclusive prefix sum of the cell particle counts: $prefix[c]$
    a. Calculated using a Blelloch scan (Chapter 4.3.2.1).
3. Cell c's particles in the newly sorted buffers will start at $prefix[c]$: for each particle that belongs to cell c, copy its data to the next available slot: this will fill $count[c]$ memory slots. The "next available memory slot" per grid cell is incremented atomically by any shader instance that wrote to its associated area in the sorted buffer.

4.3.2.1 Parallel prefix sum: Blelloch scan

Given an N length array of counts $count[0], \dots count[N-1]$, the exclusive prefix sum $prefix[k]$ is the total number of elements in all the cells preceding cell k: $prefix[k] = count[0] + count[1] + \cdots + count[k-1]$, with $prefix[0] \coloneqq 0$. A single-threaded, sequential computation of this prefix sum would iterate over the array, maintaining a running sum. The parallel Blelloch scan achieves the same result in $O(log_2 N)$ time steps using $N/2$ threads executing in lockstep, doing $O(N)$ work

overall [13]. It proceeds in two phases, working on a $s[0 \ldots N-1]$ array that must be shared between the threads.

Up-sweep (reduce) phase: Initially the elements of s are set to the cell counts. The algorithm builds a binary reduction tree over s, working from the leaves toward the root. In each step $d = 0, 1, \ldots, log_2 N$-1, letting $stride = 2^d$, each active thread at position $i = (t+1) \cdot 2 \cdot stride - 1$ (for thread index t) accumulates $s[i] += s[i - stride]$. After $log_2 N$ steps, $s[N-1]$ holds the total sum of the entire array, and every internal node of the reduction tree holds the sum of its subtree. The number of active threads halves at each step, reflecting the tree structure. For an implementation-focused array view, see Figure 8. For an algorithm-focused tree view, see Figure 9.

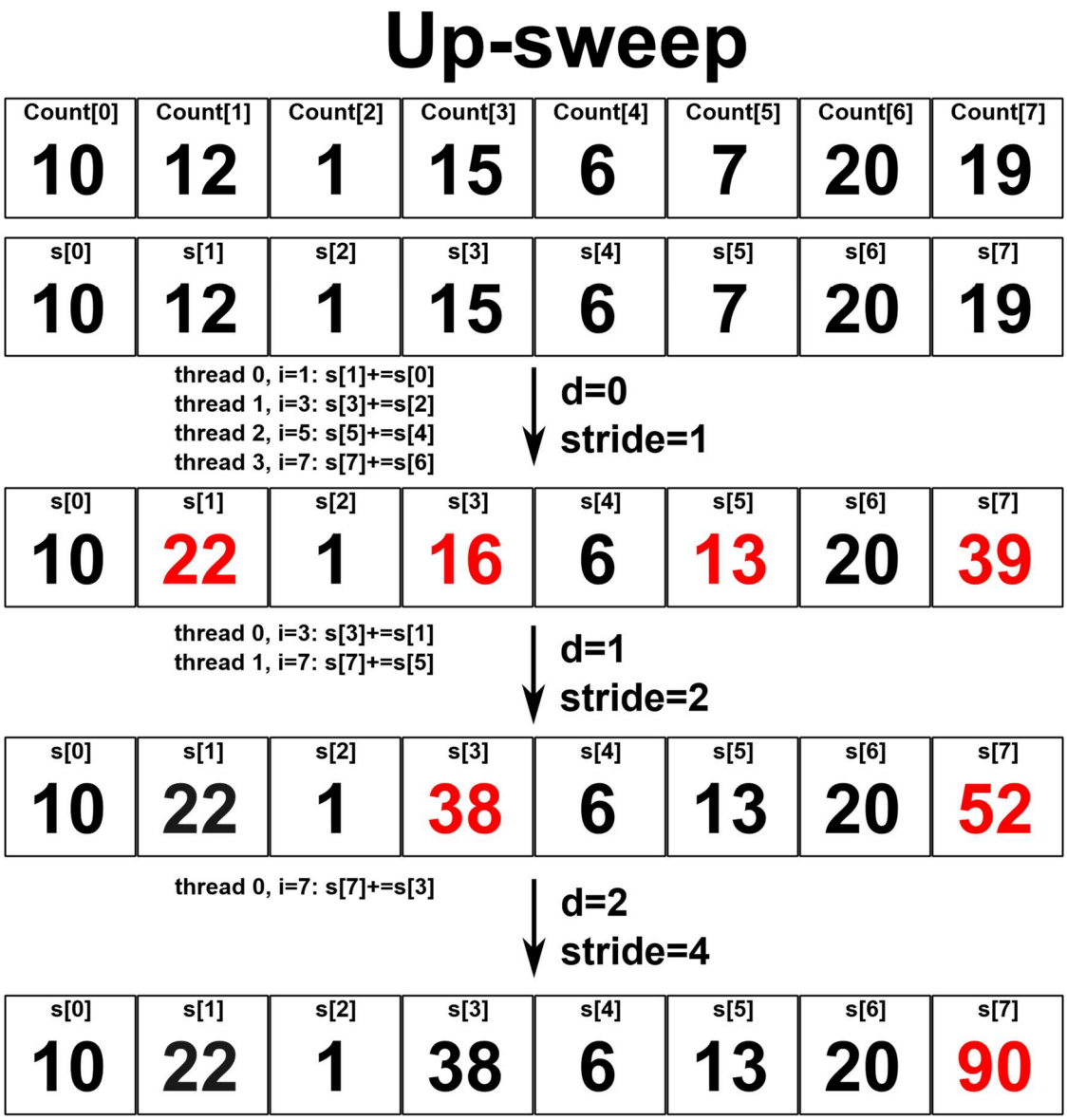

Figure 8: Blelloch scan up-sweep array visualization.

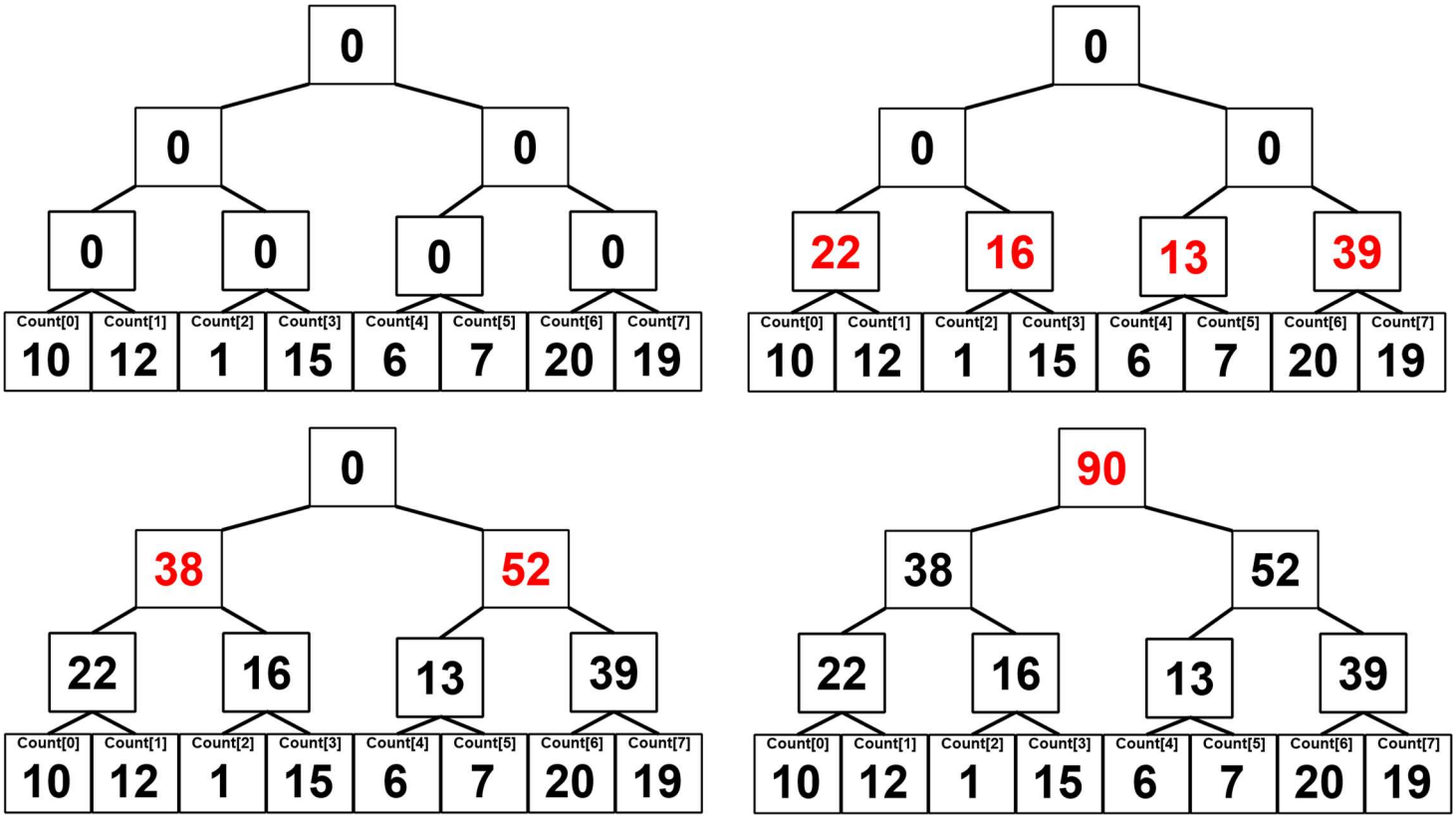

Figure 9: Blelloch scan up-sweep tree visualization.

Down-sweep phase: The exclusive prefix sum is produced by traversing the tree back down from the root. First, the root ($s[N-1]$) is set to zero, else the down-sweep would produce an *inclusive* prefix sum. Then, for each level $d = log_2 N - 1, \dots, 0$, every active thread performs a *butterfly* on a pair of nodes separated by $stride = 2^d$:

1. $left \Leftarrow s[i - stride]$: save the left child
2. $s[i - stride] \Leftarrow s[i]$: left child becomes the parent's current value
3. $s[i] \Leftarrow s[i] + left$: right child becomes parent + left child's old value

To summarize this operation: after a *butterfly*, the left child receives the parent's accumulated prefix (the sum of everything to the left of the left subtree's range), and the right child receives that same prefix plus the entire left subtree's sum (see Figure 10). After $log_2 N$ steps, $s[k]$ holds exactly $prefix[k]$. The efficiency of this approach stems from the fact that all *N*/2 threads can operate on a contiguous array in *groupshared memory*, which is a fast low-latency scratch space local to the thread group. Synchronization between steps is achieved with a single barrier instruction, which stalls all threads in the group until every thread has completed the current step, guaranteeing

that each step reads only the values written by the previous one. For an implementation-focused array view, see Figure 11. For an algorithm-focused tree view, see Figure 12.

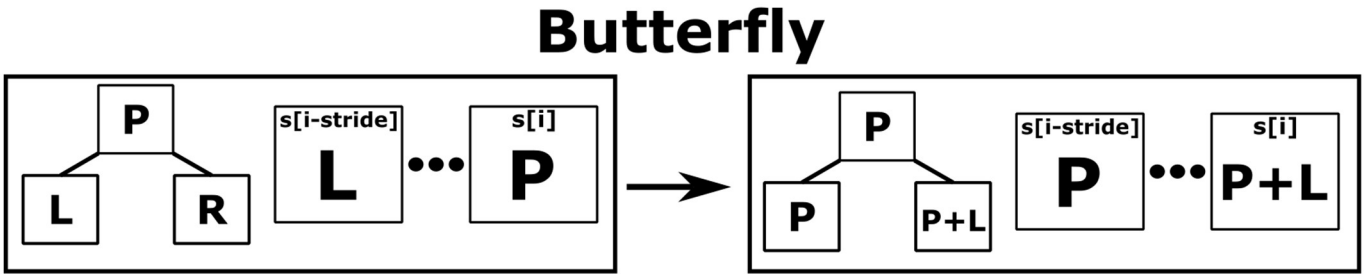

Figure 10: Blelloch scan butterfly.

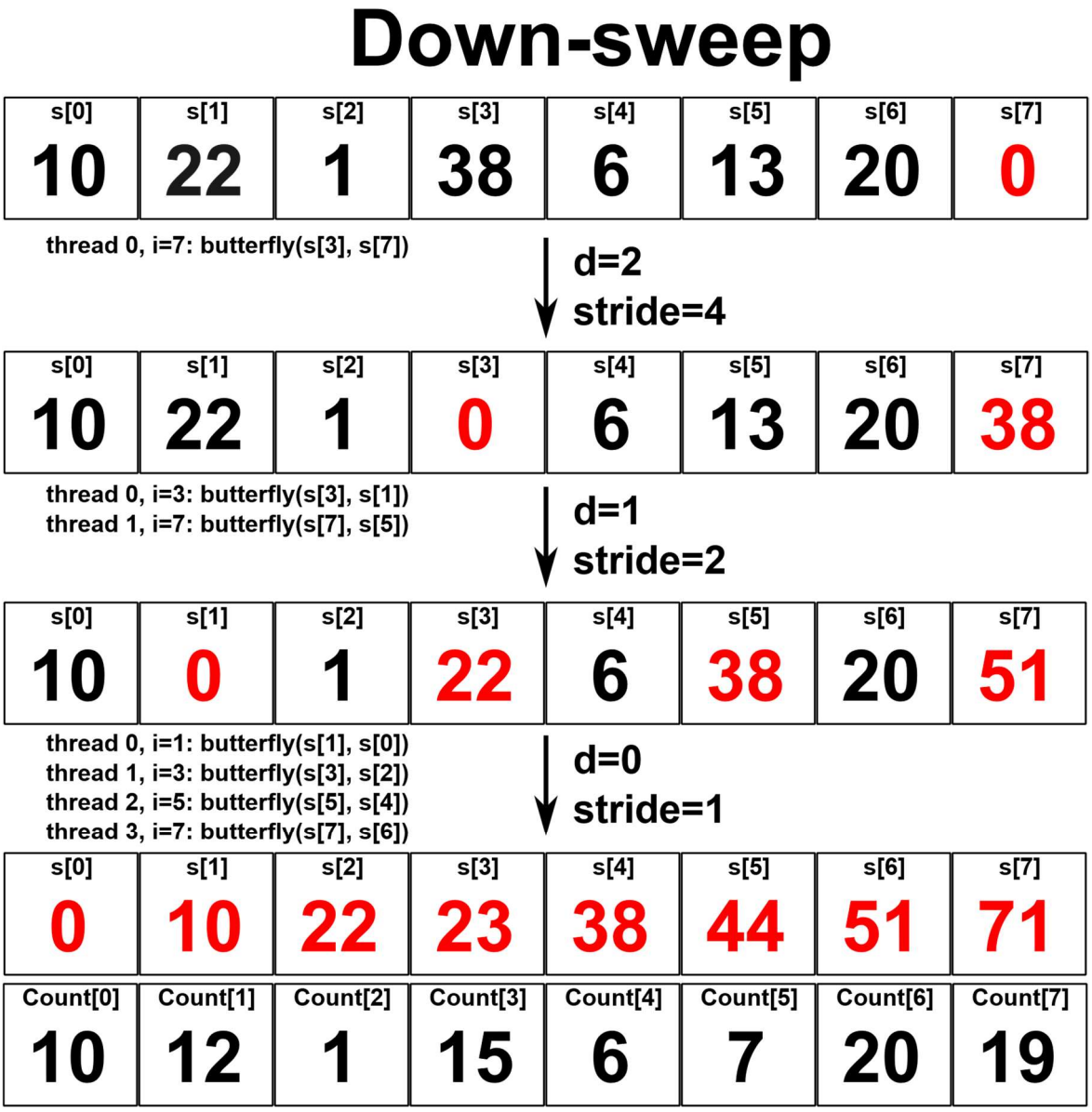

Figure 11: Blelloch scan down-sweep array visualization.

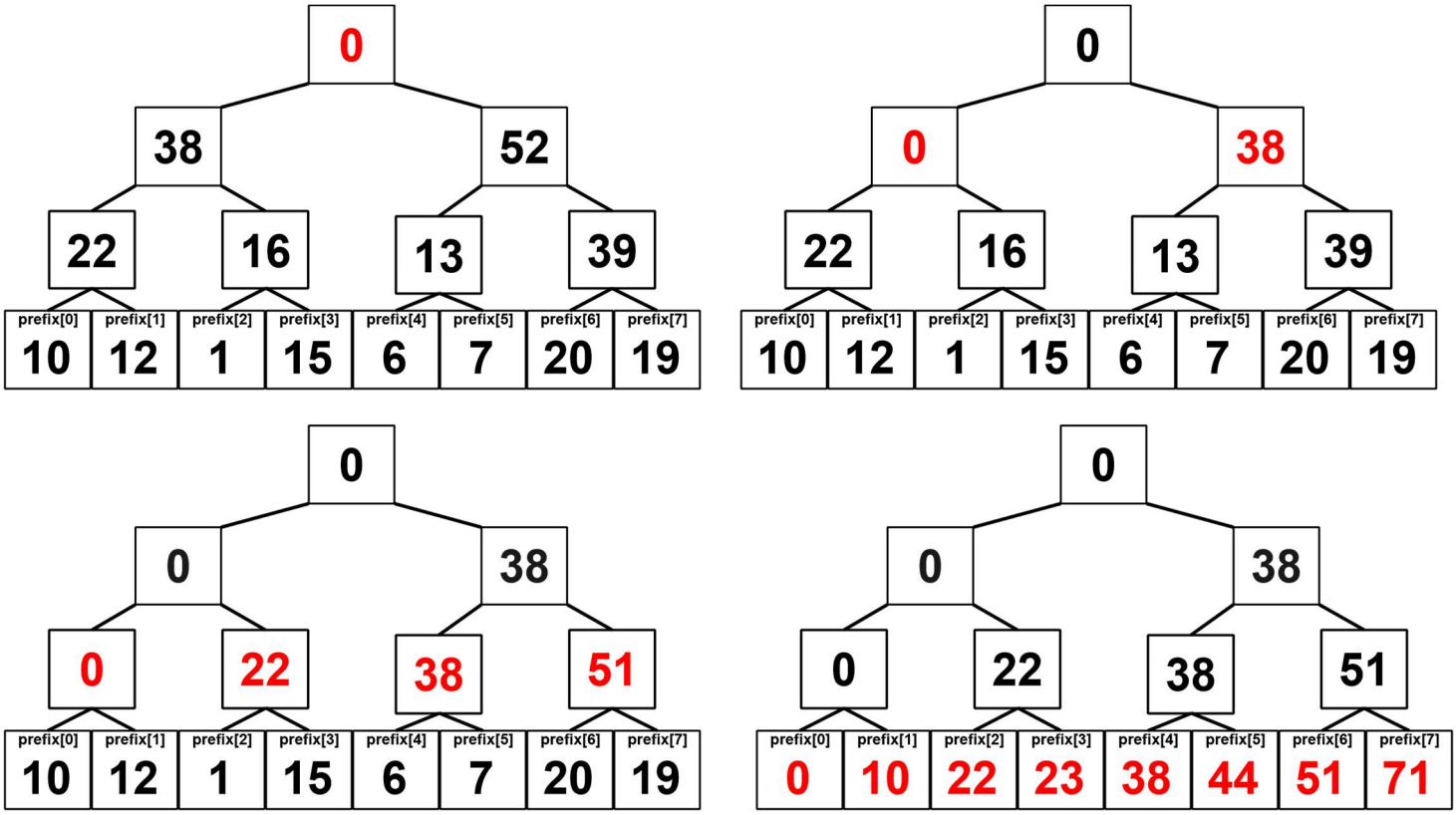

Figure 12: Blelloch scan down-sweep tree visualization.

The single-pass Blelloch scan as described above relies on all N elements being loaded into memory at once, and all $N/2$ threads operating on them within a single thread group. Two limitations arise:

1. Groupshared memory size is limited – typically 32 KB [12].
2. The number of threads in a group is capped at 1024 [14], meaning a single pass can process at most 2048 elements. In this thesis, the grid uses 128x128x128 cells, giving 2M+ cells in total – far beyond what fits in one thread group.

The solution: apply the scan hierarchically over groups of cells rather than individual cells. Each thread group processes a contiguous block of 512 cells, called a cell group. With 128^3=2,097,152 cells and 512 elements per group, there are $N = 4096$ cell groups, which still wouldn't fit in a thread group. Therefore, applying the same grouping one level up, $N = 4096$ group totals are divided into 8 super-groups of 512 elements per group each. The hierarchical scan is done in 5 passes as follows:

1. N=4096 thread groups dispatched. Each group runs a full Blelloch scan over its 512 consecutive cell counts. This produces two outputs: the intra-group exclusive

prefix sums are written to $prefix[c]$, and the group's total particle count (available as the final value of the up-sweep before it is zeroed for the down-sweep) is saved: $groupSums$. After this pass, $prefix[c]$ holds the correct prefix for cell *c within its group*, but not yet the global offset contributed by all preceding groups.

2. *M*=8 thread groups dispatched. Each super-group of 512 group totals is scanned. The intra-super-group prefix sums are saved: $groupPrefix$, and each super-group's total is saved: $superGroupSums$. After this pass, $groupPrefix[g]$ holds the correct prefix for group *g* within its super-group, but not yet the global offset from preceding super-groups.
3. Single thread group, a single Blelloch scan ran in-place over $superGroupSums$. After this pass, $superGroupSums[s]$ holds the true global offset for super-group *s*: the total particle count across all super-groups that precede it.
4. *M*=8 thread groups dispatched. Each thread group reads its super-group's global offset from $superGroupSums$, and adds it to every entry in its slice of $groupPrefix$. After this pass, $groupPrefix[g]$ holds the true global offset for cell group g.
5. *N*=4096 thread groups dispatched. Each thread group reads its cell group's global offset from $groupPrefix$ and adds it to every entry in its slice of $prefix$. After this pass, $prefix[c]$ holds the correct global exclusive prefix sum for cell *c*, i.e. the index in the sorted buffer where *c*'s particles begin.

The result is identical to what a single unlimited Blelloch scan would produce, achieved via five shader dispatches each of which stays within the groupshared memory budget and thread cap per group. Figure 13 showcases this hierarchic scan with groups and super-groups of length 2.

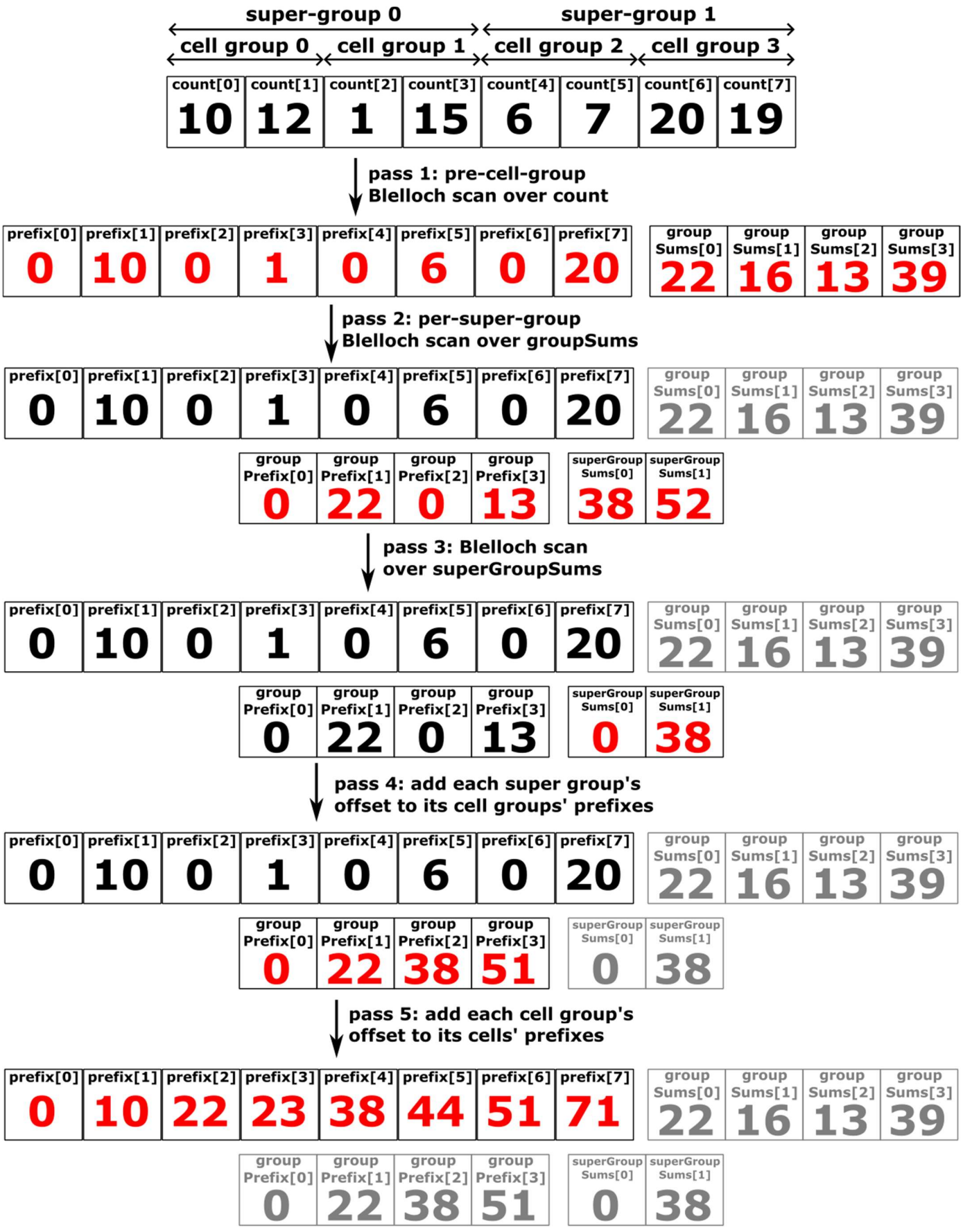

Figure 13: 5-pass hierarchical Blelloch scan.

4.3.3 Collision detection and response

Solid objects intended to displace the liquid are represented by their mesh on the rendering side, and a signed distance field on the physics side. The signed distance fields are implemented as a 3D texture, which stores for each voxel:

- The Euclidean distance to the closest point on the mesh surface, signed negative if the voxel is inside the body.
- A unit vector in the direction of the gradient of the distance field: this is the direction to move particles upon collision detection.

Efficient calculation of signed distance fields is a concept that falls outside the scope of this thesis [15], so let us assume that for solid objects that appear in the simulation, the signed distance texture is already pre-calculated. This can be loaded at initialization time, and sampled at the particle's position (after it's been transformed to the solid object coordinate space). A signed distance smaller than the particles' display radius means the particle must be moved in the gradient's direction (see Figure 14). In [1], this is done each constraint projection iteration. In addition to that, as introduced in the Adaptive Position-Based Fluids method [4], this thesis uses an additional pre-stabilization step in the prediction stage, to avoid adding undesirable velocity to low-LOD, solid-colliding particles.

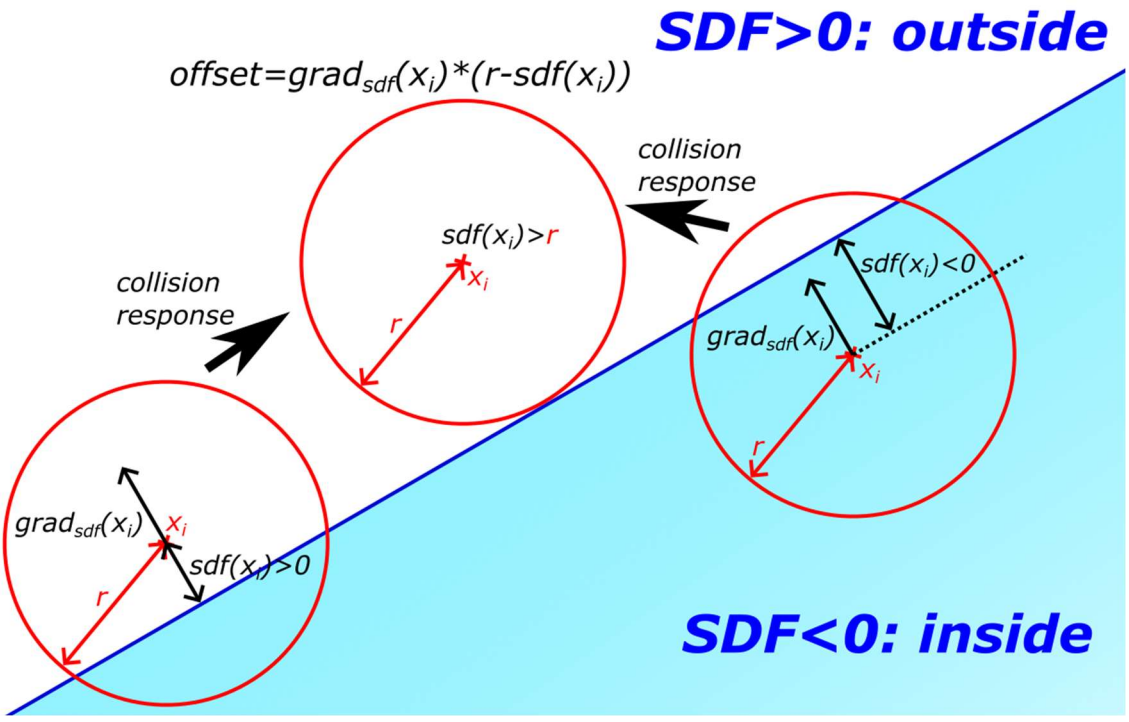

Figure 14: SDF collision detection and response.

4.3.4 Levels of Detail – Adaptive Position-Based Fluids (APBF)

Köster and Krüger introduced the concept of setting an iteration count for each particle adaptively based on its visual contribution to the scene [4]. To each particle a level of detail (LOD) gets assigned. Particles with the lowest LOD get the minimum iteration count, whereas those with the highest LOD get the maximum. Note that this requires a min-max reduction pass between calculating the per-particle LOD and assigning the iteration counts. In this thesis, as in [4], there are two ways of calculating LOD (see Figure 15):

1. Distance to the camera (DTC): particles further away get a lower LOD.
2. Distance to visible surface (DTVS), measured along a ray from the particle, the distance along that ray from the first particle hit: particles further beneath the liquid surface get a lower LOD.

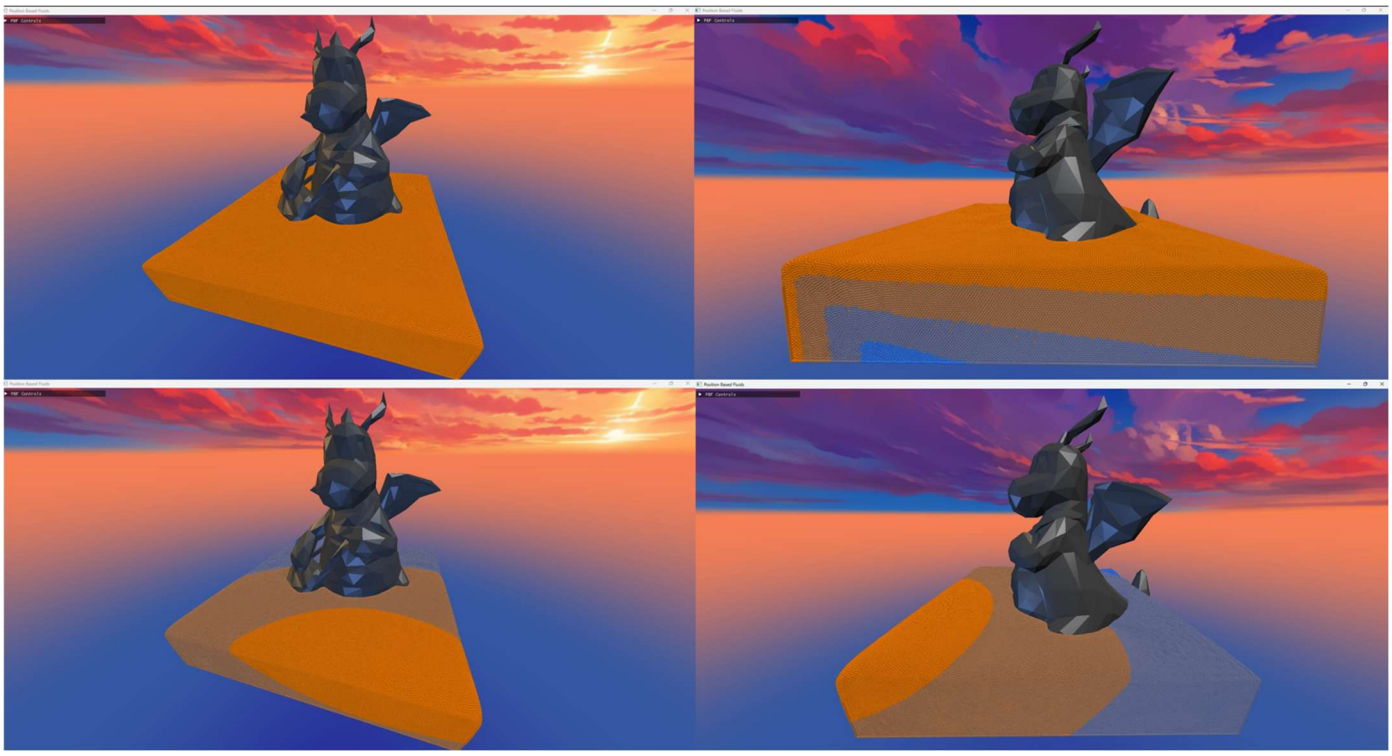

Figure 15: Particles shaded by LOD (top: DTVS, bottom: DTC), viewed from the main camera position (left), and a secondary camera position (right), to be able to see DTVS gradation.

Assigning different levels of detail to particles redistributes compute to where it matters most to the visible results. An implementation detail worth noting: DTVS calculation can be quite computation heavy. An efficient implementation used in this thesis is saving the depth data of pixels from the last render to a 2D texture. To determine

the DTVS of a particle, the texture can be sampled for the depth of the first surface along the particle's ray. The difference between the sampled depth and the particle's NDC depth is the distance to the visible surface. This method yields the LOD in normalized device units, rather than world units (like in the DTC case). This is acceptable since this information is used only in relation to other particles' LOD, which is calculated in the same units. A drawback of this method is the fact that the depth of the visible surface can only be derived from the last complete render. LOD calculations for frame N will be running concurrently with the display of frame N-1 as described in Chapter 4.1. This means that the most recent fully rendered frame whose depth data is available when calculating frame N is actually frame N-2, resulting in LOD over/underestimates at fast moving edges of the liquid, where the particles that contributed to the depth value have moved out of the ray during the past two frames. This can be seen on Figure 16.

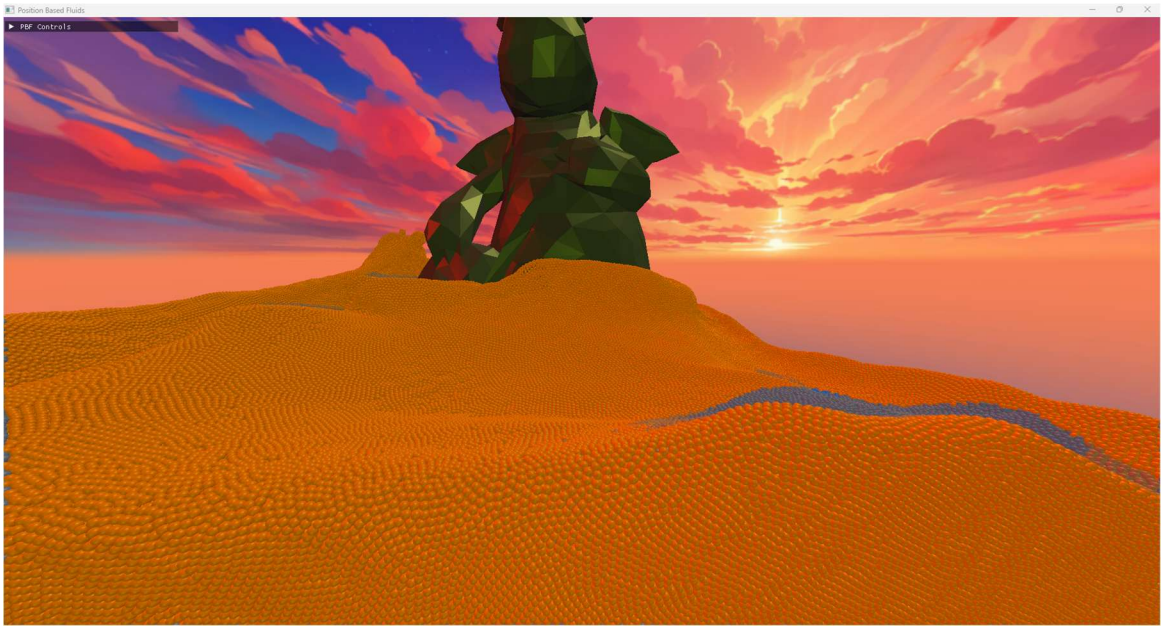

Figure 16: display for 2-frame LOD delay. Notice the rim of particles that got assigned a lower-than-expected LOD at the edge of the wave.

4.3.5 Constraint solver loop

As discussed in Chapter 3.2, the heart and soul of the PBF method is the iterative solving of the per-particle constraints, which aim to project the particles into positions that better achieve the target density. Whereas PBD can solve the constraints in a Gauss-Seidel fashion [6], Macklin & Müller use a Jacobi iteration [1]. The core PBF idea would work with both solver styles, however, a Jacobi style solver offers better GPU parallelization possibility. Another alternative could be chaotic relaxation [16], i.e.

dispatching the predicted position updates in parallel without any synchronization, such that for any given particle solving its constraint, a subset of its neighbors would already have finished their update, while some of them would still have their computations under way.

However, with the addition of adaptive iteration count in APBF, the implementation needs to use a Jacobi solver. Each iteration, particle i calculates its updated predicted position x_i^* based on the unchanged predicted position of its neighbors, and writes this data to a scratch-pad. Once all calculations for that iteration are done, the data in the scratch-pad is copied back, and the predicted positions are overwritten. However, particles with LOD l have their predicted position updated only in the first l iterations. The ordering here is important, as higher LOD particles' later iterations must run *after* the lower LOD particles' l iterations, in order to compensate for their low LOD neighbors' lack of precision. This requirement can only be respected if the iterations are run in a Jacobi fashion. Having said that, let us look at the shader dispatches that implement the constraint solver loop and the LOD calculation:

1. Calculate per-particle LOD; either DTC or DTVS.
2. Calculate the minimal and maximal LOD (reduction).
3. Set iteration count by interpolating each particle's LOD between the maximal and minimal LOD.
4. For particles that still have iterations left, update predicted position as described in Chapter 3.2.
    a. Calculate λ_i according to equation 17.
    b. Calculate Δx_i using its own λ_i and neighbors' λ_j according to equation 18. Save to scratch pad: $x_i^{scratch} \Leftarrow x_i + \Delta x_i$
    c. Once all shader threads are complete, commit the position updates stored on the scratch pad (Jacobi iteration): $x_i^* \Leftarrow x_i^{scratch}$
    d. Calculate a collision response as described in Chapter 4.3.3
    e. Decrement iteration count (some will reach 0 before others due to different LOD)

4.3.6 Velocity update and position commit

With the predicted positions finalized, they are ready to commit, but before doing so, they shall be used to calculate the actual velocity that takes internal forces into account (as opposed to the previously estimated velocity that only took into account external forces): $v_i \Leftarrow \frac{1}{\Delta t}(x_i^* - x_i)$. Adding viscosity and vorticity confinement is then a matter of modifying this velocity further, as described in chapters 3.4 and 3.5. As these calculations rely on neighboring particles' velocities, they first get written to a scratch-space in a manner similar to the Jacobi iteration for predicted positions in the previous chapter:

1. Calculate vorticity (curl of velocity vector field) according to equation 21.
2. Apply vorticity confinement and XSPH viscosity according to equations 22, 23 and 24, writing to v_i^{scrat} so that calculations use neighbor particles' data *before* the changes.
3. Commit the scratch pad once all calculations have finished: $v_i \Leftarrow v_i^{scratch}$.

With all calculations finished, positions may also be committed: $x_i \Leftarrow x_i^*$.

4.4 Graphics queue

This section discusses the rendering implementation. Each frame, the graphics queue executes the following render passes in order:

1. Background: a full-screen quad, shaded by sampling a cubemap texture.
2. Solid objects: meshes shaded using an accompanying signed distance field.
3. Depth pass for LOD: renders a particle-only scene depth to a depth texture, as discussed in chapter 4.3.4.
4. Fluid: renders either particles or a liquid surface, depending on the selected shading mode.
5. GUI pass.

The background and solid object passes involve only standard rendering techniques and are not discussed further, as this thesis is concerned with the fluid simulation aspects of the implementation.

4.4.1 GUI display

The simulation exposes several tunable parameters, such as solver iterations, adhesion, viscosity, vorticity, and artificial pressure (surface tension) coefficients. These are controlled at runtime via a graphical user interface built using Dear ImGui, a lightweight GUI library designed for use in real-time applications such as games and simulations [17]. In immediate mode, the UI is reconstructed from scratch every frame by issuing widget calls in code, rather than maintaining a persistent tree of UI objects. ImGui handles all internal state and input, and integrates with D3D12 by generating vertex and index buffers each frame which are submitted to the graphics queue for rendering on top of the scene.

Figure 17: Dear ImGui interface.

As seen on Figure 17, in the GUI, the user can choose between the following shading modes: Unicolor, Density, LOD and Liquid. The first three use particle shading, the last displays the fluid as an isosurface.

4.4.2 Particle display

The particles are displayed as billboards [18]. All their display data including their center position is double buffered, available to the shaders via shader resource views,

rather than passing them to the vertex shader in a vertex buffer. The shaders implementing impostor spheres as billboards are the following:

1. Vertex shader: extracts the data necessary for shading from the particle data buffers, then passes them on to the geometry shader.
2. Geometry shader: outputs four vertices for each input vertex. These four vertices form the corners of the two triangles that make up the camera-facing billboard.
    a. To get the camera-facing billboard's corners, we need the camera direction: $forward = cameraPos - center$.
    b. Using $worldUp = (0,1,0)$, we can get the billboard's local coordinate system: $right = worldUp \times forward, up = forward \times right$.
    c. $right$ and up are the directions in which the input vertex must be offset by ±radius to produce the corners of the billboard. Alongside the position data, the geometry shader also returns UV coordinates in the local coordinate system of the billboard.
3. Pixel shader:
    a. Discards pixels which fall outside the radius of the circle, based on the interpolated UV value of the pixel: discard, if $r^2 = u^2 + v^2 > 1.0$.
    b. To get the normal vector needed for shading, we must imagine a sphere sitting in the middle of the billboard, touching its edges. A point at UV coordinates u, v has a z-component $z = \sqrt{1.0 - r^2}$.
    c. The world-space normal needed for shading, then, can be constructed from u, v, z considering the already-derived world-space representations of the billboard's local axes: $n = u * right + v * up + z * forward$.
    d. The particles use Blinn-Phong shading with a base color determined by the shading mode:
        i. Unicolor shading uses a single color, as seen on Figure 18.
        ii. Density shading selects color based on the deviation from the desired density, going from blue (low density) to green (target density) to red (high density), as seen on Figure 19.

iii. LOD shading selects color based on the number of iterations assigned to the particle, going from orange (highest) to blue (lowest), as seen on Figure 20.

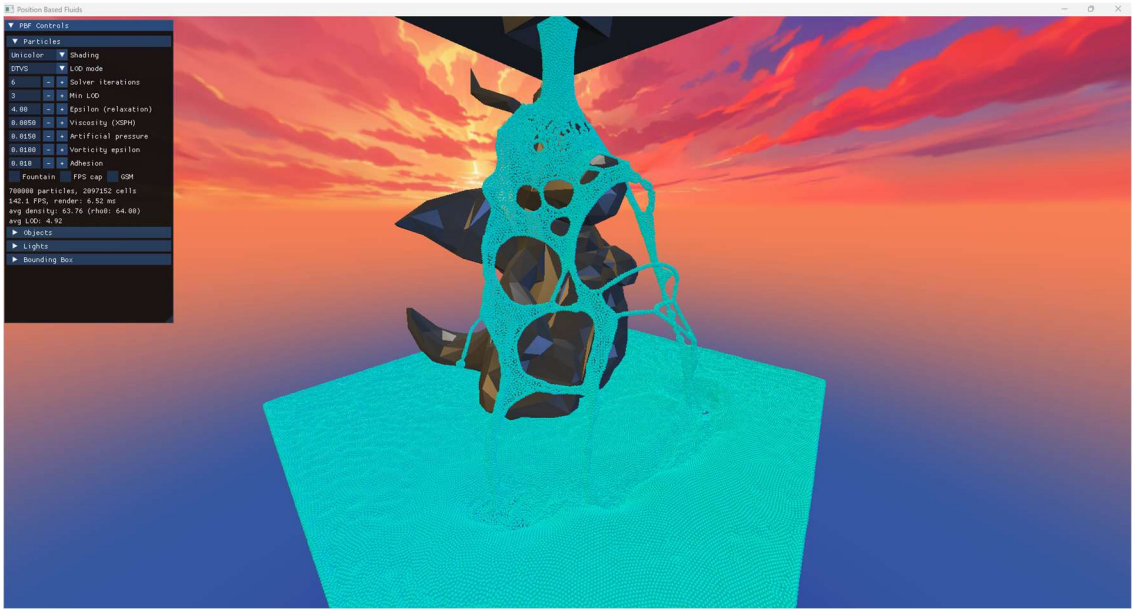

Figure 18: Unicolor shading.

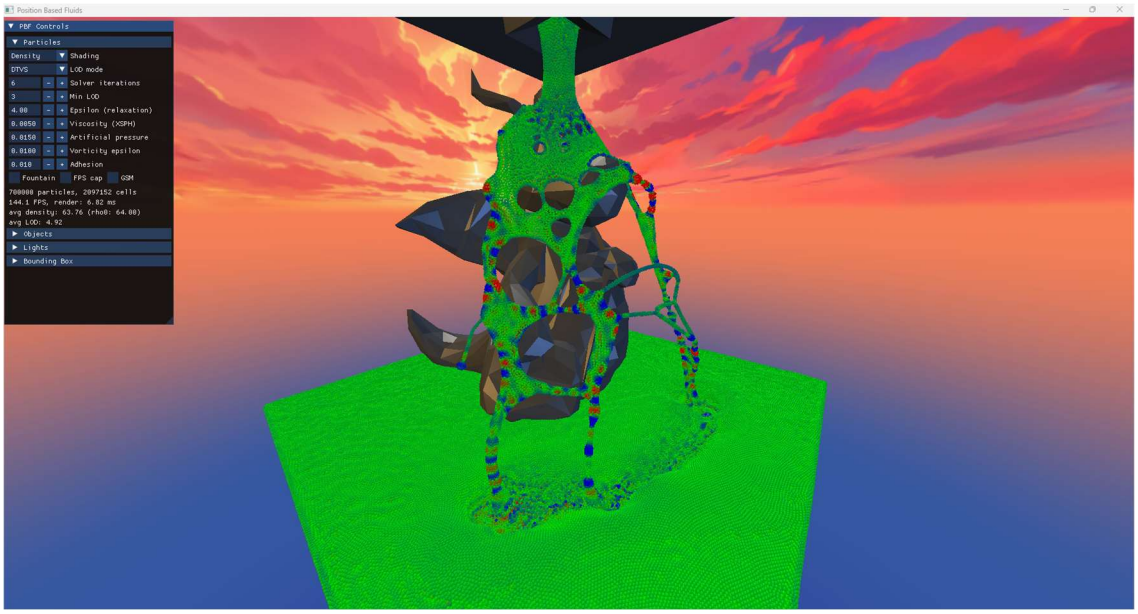

Figure 19: Density shading.

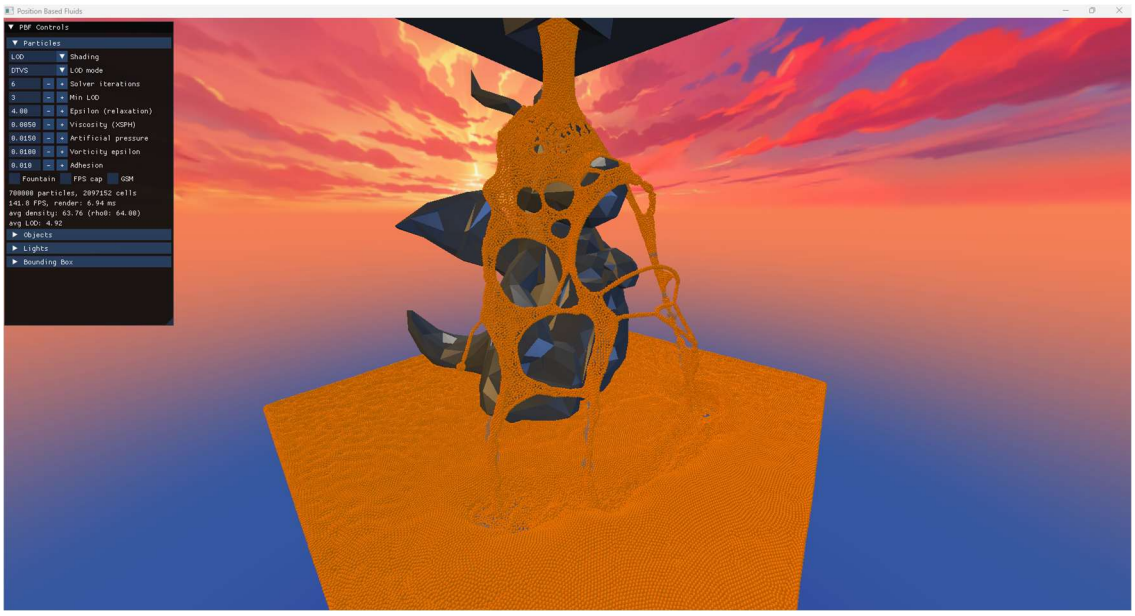

Figure 20: LOD shading.

4.4.3 Liquid shading

The final shading option is liquid shading, rendering the fluid as an isosurface. The idea is to produce a 3D texture covering the simulation area, where each voxel stores the liquid density at its center. If, based on sampling the texture, the density at a location is higher than a tunable threshold, that location is inside the liquid volume, else it's outside of it. The points where the threshold is crossed form the isosurface. The construction of the 3D density texture is a matter of sampling the PBF density at each voxel:

$$\rho_x = \sum_j W_{\text{poly6}}(x_i - x_j, h) \tag{25}$$

The naïve implementation is computationally inefficient: at each voxel, look up the neighboring particles and iterate over them. For 27 neighboring grid cells, and on average 16 particles per cell this results in 432 SPH kernel evaluations per voxel, with a voxel count of 320^3.

A more efficient approach would be starting off the texture with 0 density at each voxel, and rather than dispatching the computation per-voxel, dispatching it per-particle, splatting the already existing density values (which were calculated for shading purposes) into nearby voxels with trilinear splatting. While this approach introduces some inaccuracy in the density estimation due to the linear interpolations, it is computationally extremely cheap: Firstly, fewer threads are dispatched, as the particle count is in the hundreds of thousands, whereas voxel count for the density texture is in the tens of

millions. Secondly, SPH kernel evaluations are no longer necessary: The density values have already been calculated by the SPH solver, all that's left is splatting them into 8 nearby voxels for each particle. If we aim to preserve mathematical correctness of the SPH estimator, however, a compromise between correctness and compute is the solution implemented in this thesis: instead of trilinear splatting of the already existing density, splat the SPH smoothing kernel to neighboring voxels, utilizing the fact that the computation of the SPH density estimator is an additive task. This is mathematically identical to the naïve solution, with a fraction of the compute cost: on average, with the SPH kernel diameter being 2.5 times the size of a voxel side length, each particle will write to only ~16 neighboring voxels. The cost here comes not from the compute, but concurrent memory access. Since the compute is dispatched per-particle, concurrent threads will attempt to access the same voxel and increment it at the same time. To avoid lost updates, this must be protected against by a compare-and-swap style atomic addition, since HLSL does not natively support atomic additions on floating point values (such as the density). The computation's runtime is greatly bloated by re-runs of the compare-and-swap cycle due to concurrent voxel accesses.

Having constructed the 3D density texture and set a tunable threshold for the liquid surface cutoff, all that's left to do is identify where that threshold is crossed. This thesis uses ray marching for this:

1. If the ray intersects the bounding box, start the ray from its close face.
2. With a step size of one particle spacing (matching the voxel spacing), march the ray, evaluating the density texture at each step.
3. Should the threshold be crossed (rather than the far wall reached), refine the surface point using 6 steps of binary search.
4. Having identified the liquid surface, the surface normal required for shading is exactly the SPH density gradient at that point: $\sum_i \nabla W_{poly6}(x - x_i, h)$

To finalize the liquid shading, let us look at the shaders themselves:

1. Before the vertex and pixel shaders, a compute shader is dispatched to calculate the 3D density texture.
2. The vertex shader is fed a full screen quad and appends a ray direction to its output.

3. The pixel shader implements the ray marching above, shading the pixels in a Blinn-Phong fashion.

The results of the liquid shading can be seen on Figure 21.

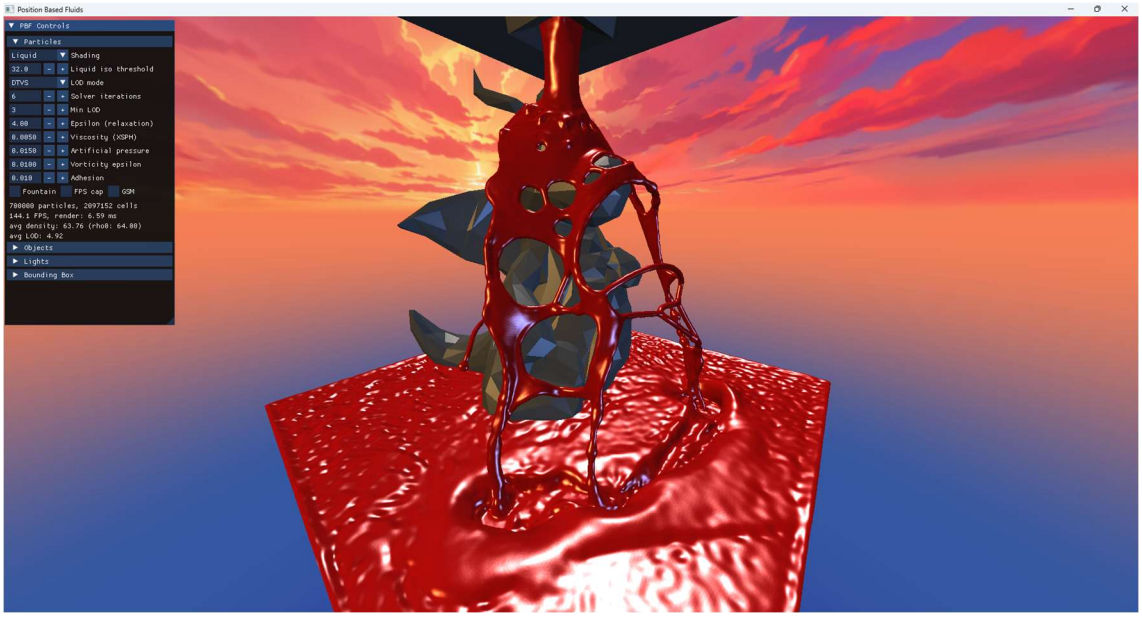

Figure 21: Liquid shading.

5 Attempts at further optimization

Beyond the optimizations already presented – parallel prefix sum, Morton codes, and adaptive PBF – two additional optimization attempts were explored that ultimately yielded no measurable fps improvement, or even led to fps degradation: The additional computational overhead offered by these changes outweighed any possible gain.

5.1 Groupshared Memory (GSM) utilization

Even after sorting the particles' data according to their cells' Morton codes, profiling still showed frequent Long Scoreboard stalls in neighbor lookups. These indicate cache misses, which I attempted to minimize by utilizing shared memory:

1. Before running computations, each thread in the thread group pre-loads its data into groupshared memory (GSM).
2. SPH neighbor lookup loops, where contribution of particles is a matter of summing over neighbors, break down into two phases:

a. GSM pass: all threads in the group iterate over the GSM in lockstep, adding the contribution of GSM-stored particles to their own local sum.
    i. These accesses are broadcast for all threads, resulting in fast data access, guaranteed to not result in a VRAM fetch.
    ii. Although Morton codes offer great spatial locality for nearby indices, there will regardless be threads in the group whose particles aren't neighbors. Their contribution will be 0 as per the rules of SPH smoothing kernel.
b. Residual pass: Particles which are potential neighbors, but their threads were not in the same thread group were skipped in the GSM pass. They are taken into account during this secondary pass, which fetches data via the regular L1-L2-VRAM path.

In practice, the overhead of the pre-load and extra 0-contributing kernel evaluations ended up being a net negative.

5.2 Sub-particle-spacing cell size

During neighbor lookups in SPH sums, the SPH kernel functions are evaluated for all potential neighbor particles. Particles are potential neighbors of others if they are located in an adjacent grid cell. Grid cells have a side length equal to the SPH kernel radius h, so that all actual neighbors are guaranteed to be among the potential neighbors. Maximizing the ratio of actual neighbors within the set of potential neighbors would lower unnecessary particle data access, as well as the number of superfluous SPH kernel evaluations that end up being 0. Using a grid cell size that is an integer fraction of h increases the ratio of actual neighbors within potential neighbors for two reasons:

1. With a finer grid cell size, cells at the corners of the potential neighbor cube can be omitted from neighbor searches, bringing the checked volume's shape closer to the sphere that contains the actual neighbors.
2. With the current scheme, the volume of the area, where potential neighbors are sought is $27h^3$, as we check all adjacent cells, each of which has volume h^3. If cells have a side length of h/n, each cell has a volume of $\left(h/n\right)^3$, however, checking only adjacent cells for potential neighbors is no longer satisfactory.

Instead, we must check cells that are n off in each direction, $(2n+1)^3$ cells overall. The volume to check for neighbors is then only $\left(2+\frac{1}{n}\right)^3 h^3$, rather than $27h^3$ (the n=1 case).

In practice, while the ratio of actual to potential neighbors improved, the finer grid required more compute overall, outweighing the gain of fewer kernel evaluations.

6 Results

The fluid simulation was tested using two kinds of hardware: NVIDIA GeForce RTX 4050 Laptop GPU and NVIDIA GeForce RTX 3080 GPU. With a grid of 128^3, particle counts in the hundreds of thousands were simulated at 35-50 fps, using the following particle parameters:

Max solver iterations	Min solver iterations	ε relaxation	Viscosity coefficient	Artificial pressure coefficient	Vorticity coefficient	Target density	SPH smoothing radius
6	3	4.0	0.005	0.015	0.01	64.0	0.625

On the NVIDIA GeForce RTX 4050 Laptop GPU, a particle count of 400k was simulated at ~40 fps. On the NVIDIA GeForce RTX 3080 GPU, a particle count of 800k was simulated at roughly the same frame rate (see Figure 22).

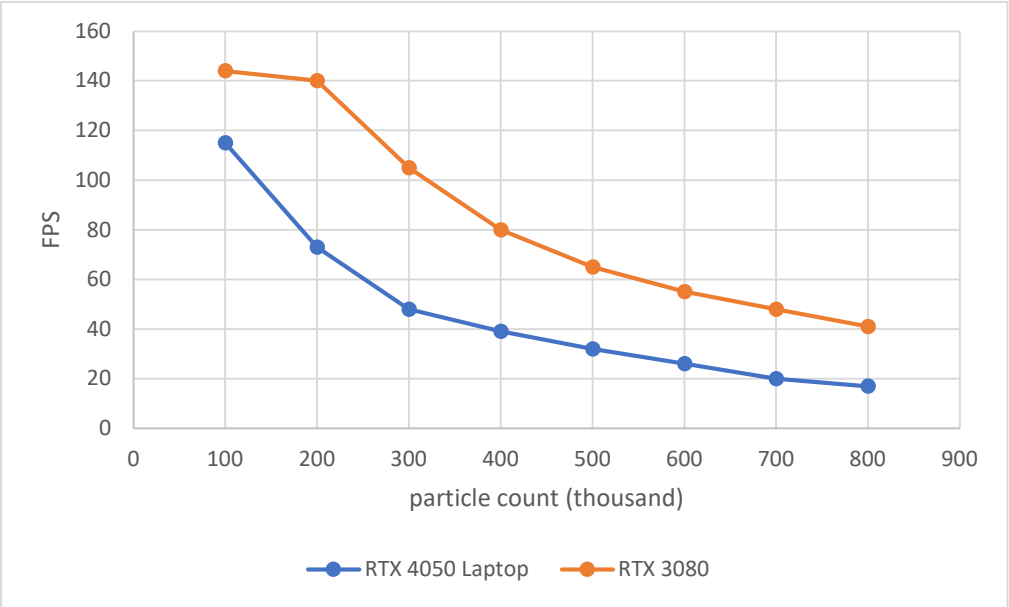

Figure 22: fps as a function of particle count.

As measured using Nvidia Nsight Graphics, on average, at 40 fps, the construction of a frame took ~35ms. This showcases the utility of the parallelism between the two command queues: even with each frame taking ~35ms to produce, we get to display a frame every ~25ms by overlapping the graphics commands for frame N-1 with the compute of frame N. The heaviest operations during the construction of a frame, in decreasing order are:

1. Calculating λ and Δx: 6x ~1-2 ms each
    a. Note that this is paid as many times as there are iterations in the constraint projection loop.
    b. Note that the first few of these passes are heavier, closer to 2ms, while the last few are lighter, closer to 1ms. This is due to the fact that the threads associated with lower LOD particles return immediately in the later iterations.
2. Calculating viscosity and applying vorticity confinement: ~3-4ms
3. Constructing the 3D density texture: ~3ms
4. Calculating vorticity ~2.5ms
5. Drawing the liquid surface ~1.5ms, of which a majority is the SPH density gradient evaluation, rather than the ray marching steps.

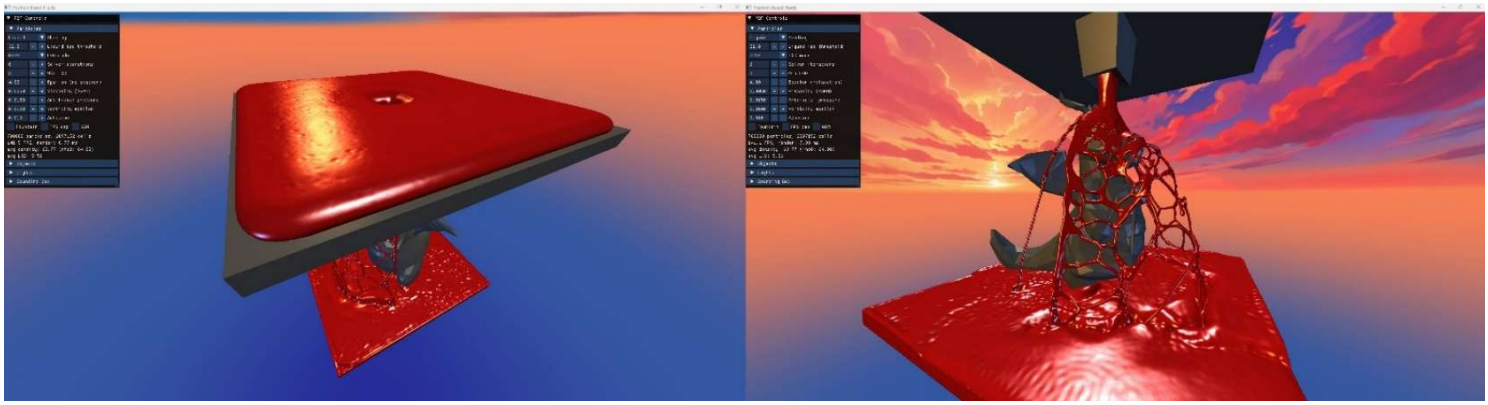

Figure 23: Liquid pouring onto a statue.

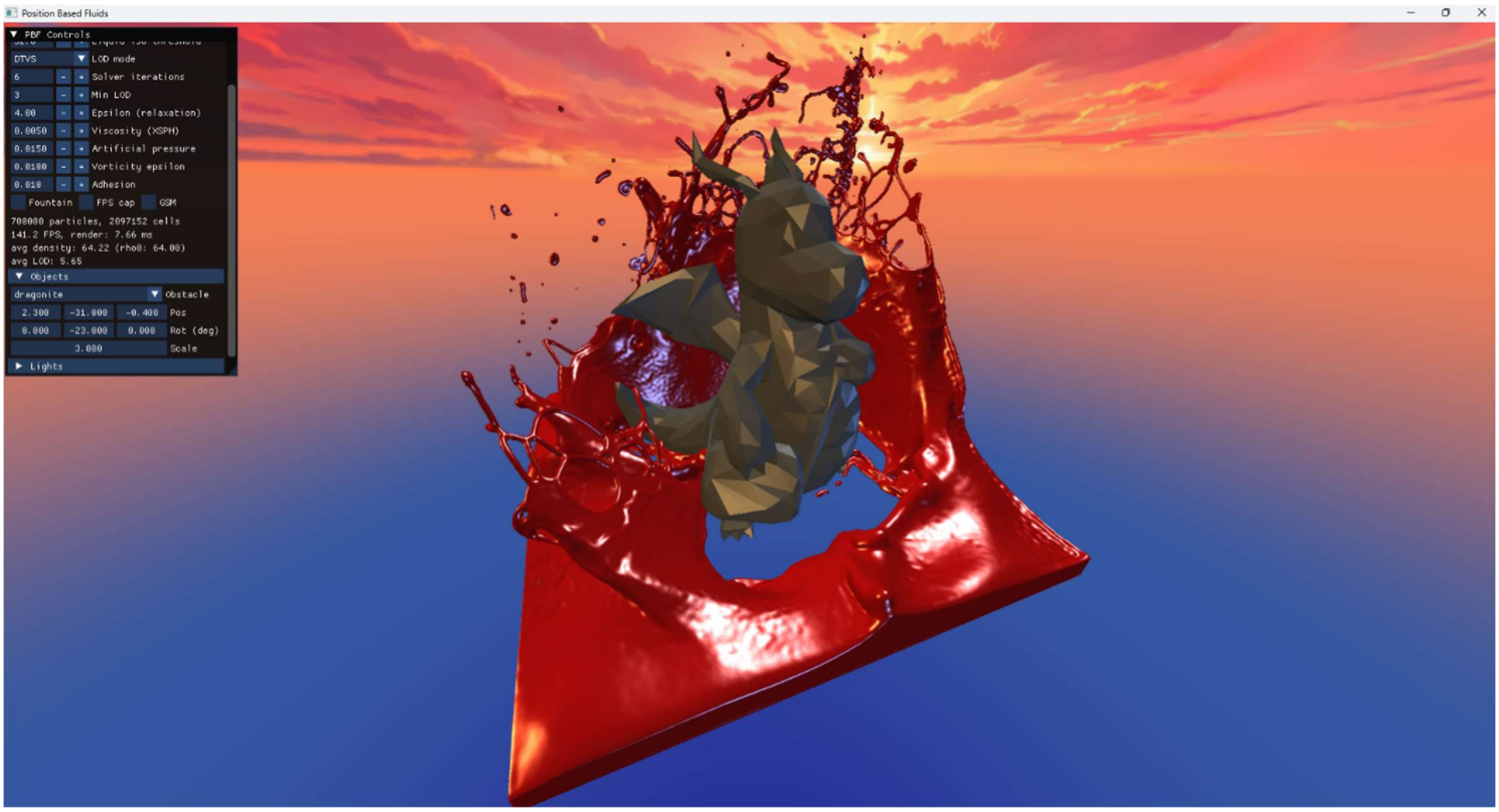

Figure 24: Liquid splashed by moving statue around.

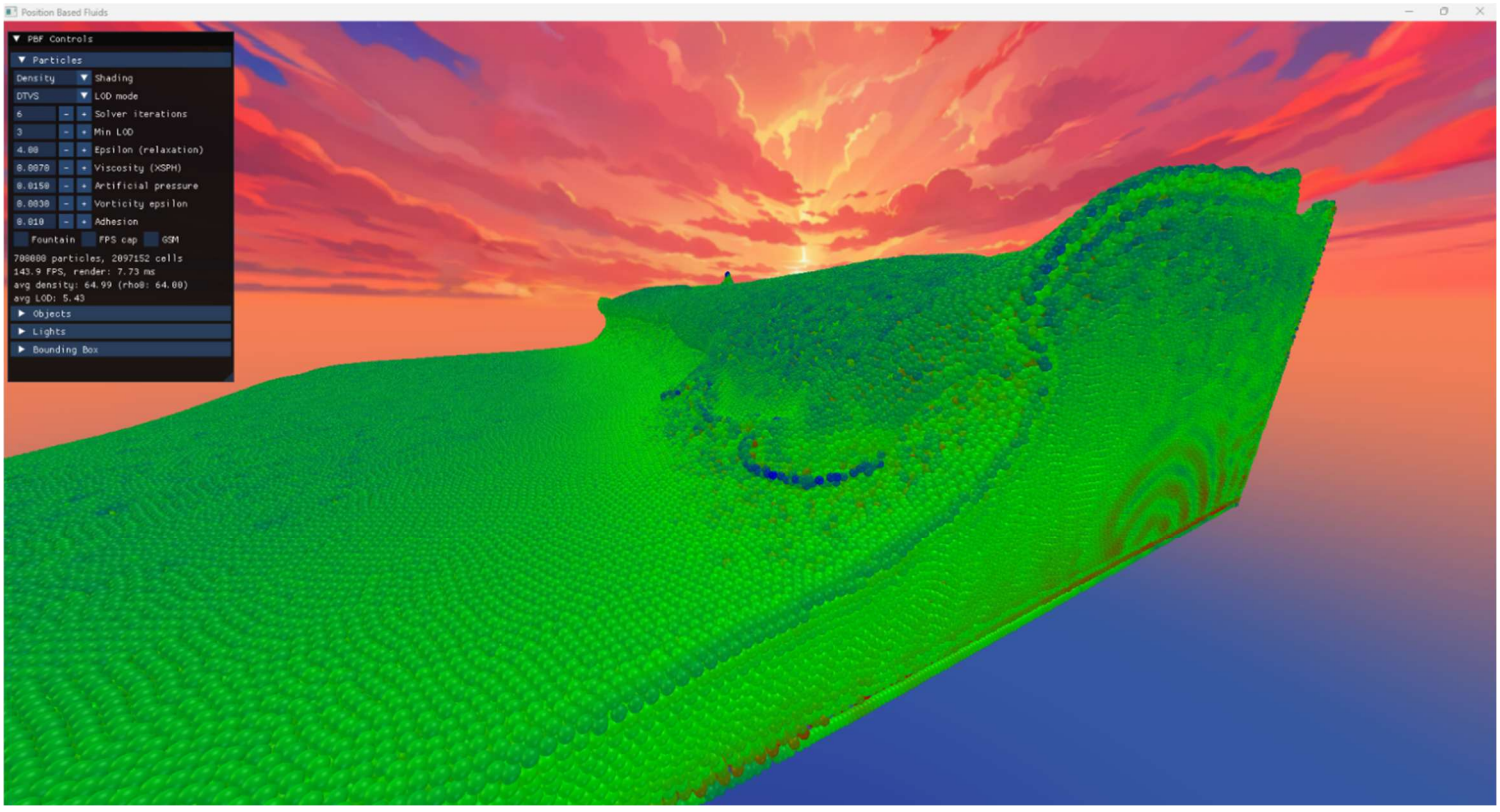

Figure 25: Wave formation, density shading.

7 Conclusion

The goal of this thesis was to design and implement a real-time fluid simulation based on the PBF method, and to demonstrate that it can produce visually compelling results even in complex, user-driven scenarios. The results validate this: stable simulation at ~40 fps was achieved with particle count in the hundreds of thousands, on both tested hardware configurations. The liquid simulation is visually rich and lifelike, owing to the artificial surface tension, vorticity confinement and XSPH viscosity (see Figure 25). Liquid-solid interaction is realistic: moving the objects or shaking the bounding box moves the fluid in a believable and natural manner (see Figure 23 and Figure 24), which would be computationally expensive with a force-based simulation. The parameter space allows the simulation of a range of fluid types, from water-like to viscous and jelly-like substances.

The density constraint was maintained with less than 2% deviation under normal conditions. Under extreme and chaotic conditions, deviations of up to 30% were observed, accompanied by a temporary fps drop due to the increased number of particles per grid cell. In such cases, recovery to target density and regular frame rate is graceful, which speaks to the robustness of the PBF framework.

The most natural direction for future work is improving the rendering to better capture the appearance of transparent liquids, such as water or oil. Achieving transparent and reflective appearance would require ray traced reflection and refraction, tracing rays both across the liquid surface and through the liquid volume to produce physically plausible results.

8 References

[1] Macklin, M., Müller, M.: *Position Based Fluid*s, ACM Transactions on Graphics (TOG), Vol. 32, No. 4, 2013, pp. 1-12

[2] Monaghan, J. J.: *Smoothed Particle Hydrodynamics*, Annual Review of Astronomy and Astrophysics, Vol. 30, 1992, pp. 543-574

[3] Müller, M., Charypar, D., Gross, M.: *Particle-Based Fluid Simulation for Interactive Applications*, Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer Animation, 2003

[4] Köster, M., Krüger, A.: *Adaptive Position-Based Fluids: Improving Performance of Fluid Simulations for Real-Time Applications*, arXiv preprint arXiv:1608.04721, 2016

[5] Solenthaler, B., Pajarola, R.*: Predictive-Corrective Incompressible SPH*, ACM SIGGRAPH 2009 Papers, 2009, pp. 1-6

[6] Müller, M., et al.: *Position Based Dynamics*, Journal of Visual Communication and Image Representation, Vol. 18, No. 2, 2007, pp. 109-118

[7] Hoetzlein, R.: *FLUIDS V.3: Interactive Million Particle Fluids*, https://ramakarl.com/fluids3/ (2026. May. 17.)

[8] Hoetzlein, R. C.: *Fast Fixed-Radius Nearest Neighbors: Interactive Million-Particle Fluids*, GPU Technology Conference, Vol. 18, 2014

[9] Tychonievich, L.: *Smoothed Particle Hydrodynamics*, University of Illinois CS 418 Course Notes, https://courses.grainger.illinois.edu/CS418/sp2023/text/sph.html (2026. May. 17.)

[10] Monaghan, J. J.: *SPH Without a Tensile Instability*, Journal of Computational Physics, Vol. 159, No. 2, 2000, pp. 290-311

[11] Fedkiw, R., Stam, J., Jensen, H. W.: *Visual Simulation of Smoke*, Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques, 2001

[12] Microsoft: *Mesh Shader*, DirectX Specs, https://microsoft.github.io/DirectX-Specs/d3d/MeshShader.html (2026. May. 17.)

[13] Harris, M., Sengupta, S., Owens, J. D.: *Parallel Prefix Sum (Scan) with CUDA*, In: GPU Gems 3, NVIDIA Corporation, https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-39-parallel-prefix-sum-scan-cuda (2026. May. 17.)

[14] Microsoft: *D3D12 Work Graphs*, DirectX Specs, https://microsoft.github.io/DirectX-Specs/d3d/WorkGraphs.html (2026. May. 17.)

[15] Erleben, K., Dohlmann, H.: *Signed Distance Fields Using Single-Pass GPU Scan Conversion of Tetrahedra*, In: GPU Gems 3, NVIDIA Corporation, https://developer.nvidia.com/gpugems/gpugems3/part-v-physics-simulation/chapter-34-signed-distance-fields-using-single-pass-gpu (2026. May. 17.)

[16] Chazan, D., Miranker, W.: *Chaotic Relaxation*, Linear Algebra and Its Applications, Vol. 2, No. 2, 1969, pp. 199-222

[17] Cornut, O.: *Dear ImGui: Bloat-free Graphical User Interface for C++ with Minimal Dependencies*, GitHub repository, https://github.com/ocornut/imgui (2026. May. 17.)

[18] Paroj: Chapter 13. *Lies and Impostors, Learning Modern 3D Graphics Programming*, https://paroj.github.io/gltut/Illumination/Tutorial%2013.html (2026. May. 17.)

9 Appendix / Függelék

9.1 Nyilatkozat generatív mesterséges intelligencia alkalmazásáról

☐ **Nem használtam** semmilyen generatív MI segédeszközt.

☑ **Használtam** generatív MI segédeszközt. Az MI-vel generált tartalmakat ellenőriztem, a generált kimenetek valóságtartalmáról meggyőződtem, az alábbi táblázatban megfelelően jelöltem minden használatot.

Felhasználási módok	**Generatív MI eszköz(ök) neve**	**Érintett részek** (fejezet, oldalszám, hivatkozás)	**Használat becsült aránya** (felhasználási módonként)
Irodalomkutatás	Claude	Literature overview	<1%
Prompt lényegi része	I'm working on a thesis relating to particle-based fluid simulation. Recommend research articles that might be of interest. Help me interpret them.		
Programkód generálása	Github Copilot, Claude Code	Algorithm & Implementation	<5%
Prompt lényegi része	What arguments does this DX12 function take? How do I create a shader visible descriptor heap? What does this compiler error mean?		
Új ötletek, megoldási javaslatok generálása	Claude, Claude Code	Algorithm & Implementation	<5%
Prompt lényegi része	Profiling shows X, Y, and Z bottlenecks in the simulation's runtime. Recommend some ideas to alleviate it. How would you reorganize this class to improve readability?		
Vázlat létrehozása (szövegstruktúra, vázlatpontok)	-	-	-
Prompt lényegi része	-		

Szövegblokkok létrehozása	Claude	Introduction, Literature overview, Results, Conclusion	<5%
Prompt lényegi része	I'd like a sentence that forms a logical transition between paragraphs X and Y. Re-write this sentence to be easier to understand.		
Képek generálása illusztrációs célból	-	-	-
Prompt lényegi része	-		
Adatvizualizáció, grafikonok generálása adatpontok alapján	-	-	-
Prompt lényegi része	-		
Prezentáció készítése	-	-	-
Prompt lényegi része	-		
Egyéb (nevezze meg) API dokumentációk értelmezése	Claude	Algorithm & Implementation	<1%
Prompt lényegi része	Explain what a compute shader is in DX12! What is the difference between an SRV and an UAV?		
Összesített százalékos érték (a feladat érdemi részére nézve)			**1-2%**
Összesített érték rövid, szöveges indoklása: **Az MI eszközök hozzájárulása a feladat során főleg asszisztensi (kód autocomplete), hibakereső, és dokumentáció értelmező. A leggyakoribb felhasználás az utóbbi, gyakorlatilag internetes keresések kiváltása, amelyet nehéz számszerűsíteni, hiszen nincs konkrét százalékos befolyása sem a kódra, sem az írásműre.**			